

Module 1

Module#	CO	Topics	Hours
Module-1	CO1	Introduction to database systems: Basic concepts and definitions, threeschema architecture, data independence, Concept of data models, types of data models, database languages, integrity, database users, Entity- Relationship model, Constraints & Keys, Extended Entity Relationship model, Relational model, Mapping of E-R model to relational schema, System structure of DBMS, Codd's 12 Rules.	12
Module-2	CO2	Query languages: Relational Algebra, basic operations, join operations, grouping & aggregation, Relational Calculus; Query processing and optimization: Evaluation of relational algebra expressions, Heuristic-based Query optimization.	11
Module-3	CO3	Database design: Functional dependencies, Armstrong axioms, Attribute closure, Equivalence sets of FD, Minimal cover; Normalization: Dependency & attribute preservation, lossless join; Normal Forms: 1NF, 2NF, 3NF, BCNF, Testing for lossless design, Multi-Valued Dependency (MVD), 4NF and 5NF.	11
Module-4	CO4	Transaction processing: Basic concepts, ACID Properties, Serializability, Concurrency Control Schemes – lock-based & timestamp-based protocols, Deadlock handling, deadlock prevention, detection and recovery; Database Recovery: types of database failures, Recovery techniques - log-based recovery, checkpoints, shadow paging.	12
Module-5	CO5	Storage strategies: Storage Architecture, File and Record Organization, Types of Indexes, B-Tree, B+ Tree, Index Files, Hashing, Data Dictionary; Distributed databases: Homogeneous vs. heterogeneous, Fragmentation & replication, Data transparency; Introduction to NoSQL: Properties, Columnar families, different NoSQL systems.	10
		Total	56 Hours

Books

- **Text Book**

- Database System Concepts, A. Silberschatz, H. F. Korth, and S. Sudarshan, McGraw-Hill,
- Fundamentals of Database Systems, R. Elmasri and S. B. Navathe, Pearson Education
- SQL, PL/SQL - The Programming Language of Oracle, I. Bayross, BPB Publications

- **Reference Book**

- Database Management Systems, R. Ramakrishnan and J. Gekhre, McGraw Hill Education
- Database Management Systems, R. P. Mahapatra and G. Verma, Khanna Publishing
- Introduction to Database Systems, C. J. Date, Pearson Education

Basic Concepts

- **Data** - Collection of facts and figures
- **Information** - Processed data
- **Database** - Collection of ***related data*** in defined order
ex- student database, employee database
- **DBMS** - **Database + Set of Programs**
- **DBMS** contains information about a **particular enterprise**
 - It contains **interrelated data**
 - **Set of programs** to access the data
 - An environment that is both **convenient and efficient** to use.

(It is a software system that facilitates the processes of ***defining, constructing, manipulating and sharing databases*** among various users and applications.)

Ex- **MySQL, MS SQL Server, Oracle, DB2, Microsoft Access**, etc.
- **Goal** - provide a way to Store & Retrieve data from the database in a Convenient & Efficient manner.
- **Database System** = Database + DBMS s/w
A collection of interrelated data and a set of programs to access and modify these data.

Understanding Data and Information

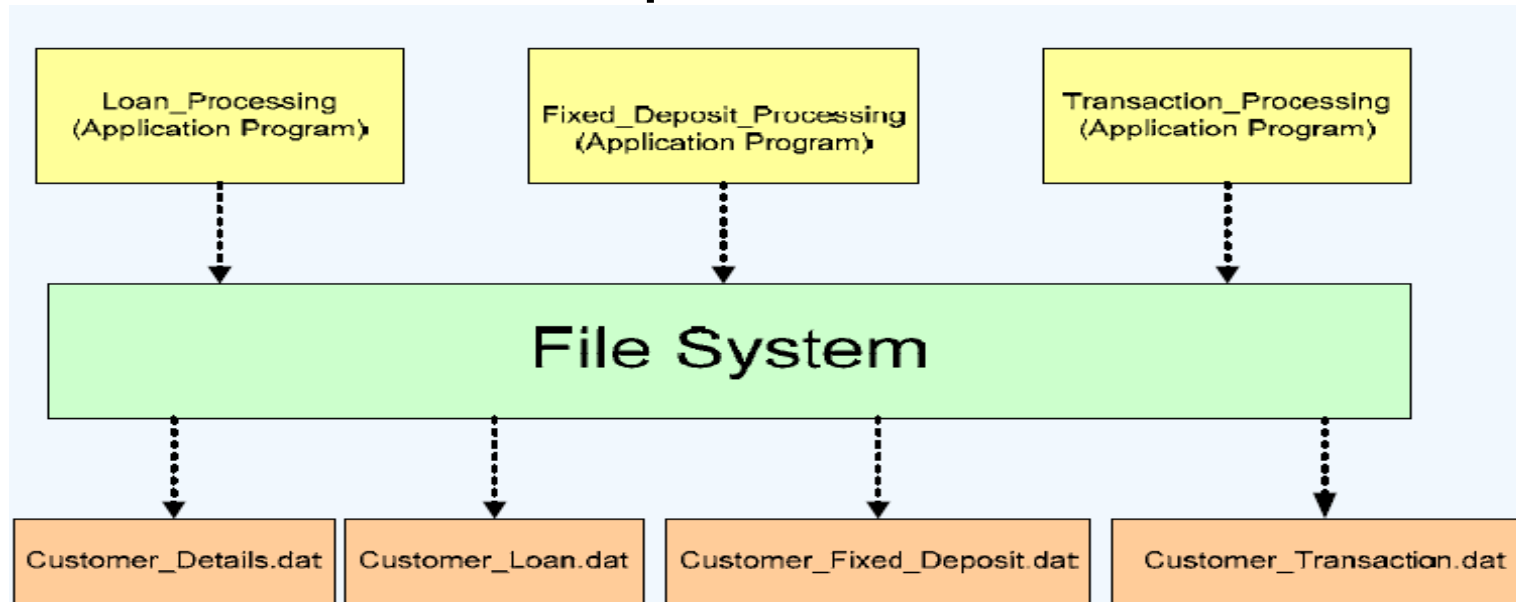
Data	Information
Data is raw facts and figures	Processed form of data
23 is data	Age 23 is information
Data cannot be used for decision making	Information is useful for decision making
Data is atomic.	Information is collection of data.

Maintaining Database on Computers

- There are two approaches
 - Traditional File Based Approach
 - Centralized DBMS approach

File Based Approach

- It is an early attempt to computerize the manual filing system.
- Data is stored in flat files. For example: Text files, csv files
- Each file called a flat file, contained and processed information for one specific function.



Drawbacks of File Processing System

- **Student db system included in a file**
 - **General info**(regd no, Name, dob, branch, address, Tel-phno, marks)
 - **Hostel info** (regd no, Name, dob, branch, address, Tel-phno, mess bill, room no)
 - **Library info**(regd no, name, dob, branch, address, Tel-phno, date-of-issue, date-of-return, no of books issued, fine)
 - **Account info** (regd no, name, dob, branch, address, Tel-phno, year-of-joining, Fee, instalment, Balance, Total)
- **The** file system contain some common data of students like **regd no, Name, dob, branch, dob, address, Tel-phno** causing redundant info and wastage of storage space.
- But in centralized DBMS system, this can be eliminated in a great extend as data can be shared by many number of application.
 - **General info**(regd no, Name, dob, branch, address, Tel-phno, marks)
 - **Hostel info** (regd-no, mess bill, room no)
 - **Library info**(regd no, , date-of-issue, date-of-return, no of books issued, fine)
 - **Account info** (regd no, year-of-joining, Fee, instalment, Balance, Total)
- Data need not be stored repeatedly as **all applications** can access the information by joining the relation on basis of common column.

Limitations of the File-Based Approach

long or
short

❑ Distinguished and Isolated Data:

Imagine a user needs information that is not possible to be provided using a single file, multiple different files were required, which are situated in different departments. So all the employees first need to manually and carefully check each of the files in each department and find the relationship between them to extract the information that the user wants.

❑ Duplication of data and data inconsistency

Due to the manual storing of data, the same data used to be present in multiple locations using the space in each of the Hard Disk, below are some following reasons for which Data Redundancy can be a major issue Storing the same data multiple times not only wastes resources in every machine but also is costly to maintain and wastes time.

❑ Data integrity can't be enforced

Loss of data integrity is another major issue of Data Redundancy, imagine someone's address is present in multiple systems and he has applied to update the address, in one system the address gets updated but in the rest of them it remains the same, so the if someone from any different department where the data is not updated tries to send them any letter or something then it would go to a wrong address.

❑ Data Dependence

Files and information were stored in a certain specific format in files which is hard coded by programmers in languages like C/C++, COBOL, etc. So if any of the file's format changes then the programmers need to update the code every time and the format of every piece of data stored in that file will be changed, which is a rigorous task for programmers.

❑ Incompatible file formats

The file structure would vary based on the programming language that was used to store them, for example, if a system used COBOL to store certain data then the structure of that data would differ from the one written in C/C++.

- ❑ Multiple user access at the same time is not possible

Two or more users can view the same file simultaneously, but the problem arises when they try to update the same file simultaneously.

- ❑ Less provision of back up and recovery

If any failure occurs in any file it is difficult to recover.

- ❑ Less secure

- ❑ Atomicity is not allowed

A computer system, like any other device, is subject to failure. In many applications, it is crucial that, if a failure occurs, the data be ***restored to the consistent*** state that existed ***prior to the failure***.

Ex: Fund Transfer

All the operations in a database must be atomic- Either the operation **entirely executed** or **none** of the steps of the operation should be executed. It is difficult to ensure atomicity in a conventional file-processing system.

Database Approach

- Databases provide a centralized location for storing data, which makes it easier to manage and access.
- This is especially important for organizations that have a large amount of data or that need to store data for an extended period of time.

Example: File Based System

General Office	Library	Hostel	Account Office
Rollno Name Class Father_Name Date_of_birth Address Phone_No Previous_Record Attendance Marks etc.	Rollno Name Class Address Date_of_birth Phone_No No_of_books_issued Fine etc.	Rollno Name Class Father_Name Date_of_birth Address Phone_No Mess_Bill RoomNo etc.	Rollno Name Class Address Phone_No Fee Installments Discount Balance Total etc.

Example: Database Approach

General Office	Library	Hostel	Account Office
Rollno Name Class Father_Name Address Phone_No Date_of_birth Previous_Record Attendance Marks etc.	Rollno No_of_books_issued Fine etc.	Rollno RoomNo Mess_Bill etc.	Rollno Fee Installments Discount Balance Total etc.

Characteristics of DBMS

1) Self-describing nature of a database system:

A DBMS **catalog** stores the *description* of the database. The *description* is called **meta-data**. This allows the DBMS software to work with different databases.

2) Data Abstraction:

Application Program and structure of data files are independent in case of DBMS(**Program-Data Independence**)

A **data model** is used to hide storage details and present the users with a *conceptual view* of the database.

Support of multiple views of the data:

Each user may see a different view of the database, which describes *only* the data of interest to that user.

TRANSCRIPT

Student_name	Student_transcript				
	Course_number	Grade	Semester	Year	Section_id
Smith	CS1310	C	Fall	08	119
	MATH2410	B	Fall	08	112
Brown	MATH2410	A	Fall	07	85
	CS1310	A	Fall	07	92
	CS3320	B	Spring	08	102
	CS3380	A	Fall	08	135

COURSE_PREREQUISITES

Course_name	Course_number	Prerequisites
Database	CS3380	CS3320
		MATH2410
Data Structures	CS3320	CS1310

4) Sharing of data and multiuser transaction processing :

Allowing a set of concurrent users to retrieve and to update the database. Concurrency control within the DBMS guarantees that **each transaction is correctly executed or completely aborted**. OLTP (Online Transaction Processing) is a major part of database applications.

DBMS

- It is a software for managing the database.
- It acts as an interface between the user and the database.
- DBMS software includes: MySQL, Oracle, SQL Server, IBM DB2 etc.

Functions of DBMS:

- Constructs the database
- Manipulates the database
- Provides back up and recovery strategies
- Maintains integrity and security of data
- Provides maintenance for longer period of time

Advantages of DBMS

- Reduce redundancy
- Inconsistency can be avoided
- Integrity can be maintained
- Data can be shared
- Restricting unauthorized access
- Backup and recovery

Schema versus Instance

- Database Schema:
 - The **description** of a database.(Overall design of the db)
 - Includes descriptions of the database structure, data types, and the constraints on the database.
 - **Does not change frequently**
 - The conceptual organization of the database as viewed by the DBA
- Schema Diagram:
 - An **illustrative** display of (most aspects of) a database schema.

Schema versus Instance

- Database state:
 - The actual data stored in a database at a particular moment in time.
 - Collection of information stored in the db at a particular moment.
 - Also called database instance (or occurrence or *snapshot* or *extension* of the scheme).
 - Instances are changed frequently(insertion, deletion and modification)
 - Every time we insert or delete a record or change the value of a data item in a record, the database moves from one state to another but schema is fixed.
 - The database schema changes very **infrequently**.
 - The database state changes every time the database is **updated**.

Schema versus Instance

- **Initial Database State:**

- Refers to the database state when it is initially loaded into the system.

- **Valid State:**

- A state that satisfies the structure and constraints of the database
 - The DBMS is responsible for ensuring that every state of the database is a valid state.

Example of a database Schema

STUDENT

Name	Student_number	Class	Major
------	----------------	-------	-------

COURSE

Course_name	Course_number	Credit_hours	Department
-------------	---------------	--------------	------------

PREREQUISITE

Course_number	Prerequisite_number
---------------	---------------------

SECTION

Section_identifier	Course_number	Semester	Year	Instructor
--------------------	---------------	----------	------	------------

GRADE_REPORT

Student_number	Section_identifier	Grade
----------------	--------------------	-------

Example of a database state

STUDENT

Name	Student_number	Class	Major
Smith	17	1	CS
Brown	8	2	CS

COURSE

Course_name	Course_number	Credit_hours	Department
Intro to Computer Science	CS1310	4	CS
Data Structures	CS3320	4	CS
Discrete Mathematics	MATH2410	3	MATH
Database	CS3380	3	CS

SECTION

Section_identifier	Course_number	Semester	Year	Instructor
85	MATH2410	Fall	07	King
92	CS1310	Fall	07	Anderson
102	CS3320	Spring	08	Knuth
112	MATH2410	Fall	08	Chang
119	CS1310	Fall	08	Anderson
135	CS3380	Fall	08	Stone

GRADE_REPORT

Student_number	Section_identifier	Grade
17	112	B
17	119	C
8	85	A
8	92	A
8	102	B
8	135	A

PREREQUISITE

Course_number	Prerequisite_number
CS3380	CS3320
CS3380	MATH2410
CS3320	CS1310

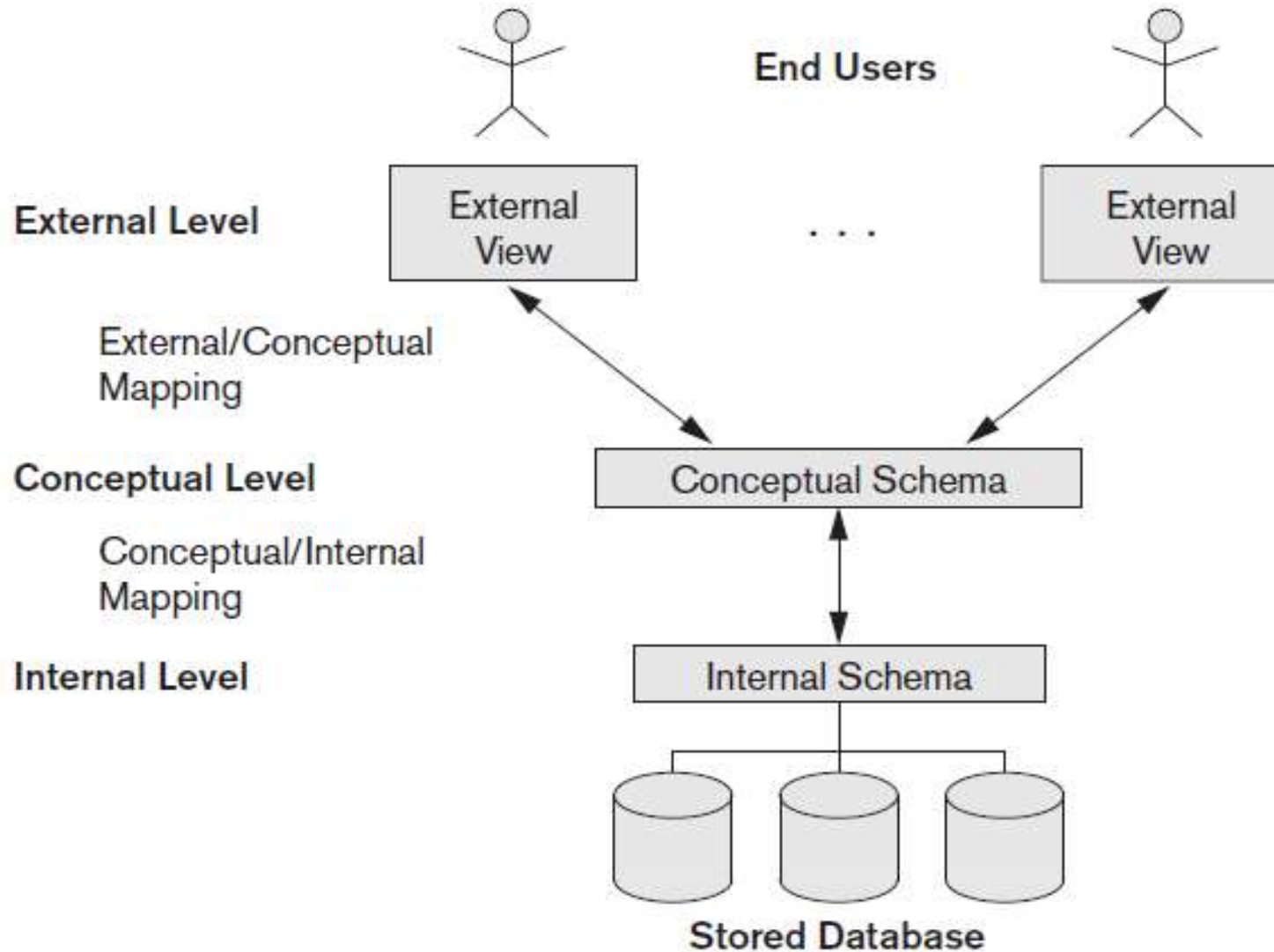
Three-Schema Architecture

- The goal of the three-schema architecture is to separate the user from the physical database.
- Also known as **ANSI/SPARC (American National Standards Institute/ Standards Planning And Requirements Committee) architecture**.
- Hide the complexity from users through several levels of abstraction, to simplify user's interactions with the system.
 - **External or view level**
 - **Conceptual or logical level**
 - **Physical level**

Objective of the Three Level Architecture

- Each user should be able to access the same data, but have a different customized view of the data.
- User's interaction with the database should be independent of storage considerations.
- The Database Administrator (DBA) should be able to change the database storage structures without affecting the user's views.
- The internal structure of the database should be unaffected by changes to the physical aspects of storage, such as the changeover to a new storage device.
- The DBA should be able to change the conceptual structure of the database without affecting all users.

Three Level Architecture of DBMS

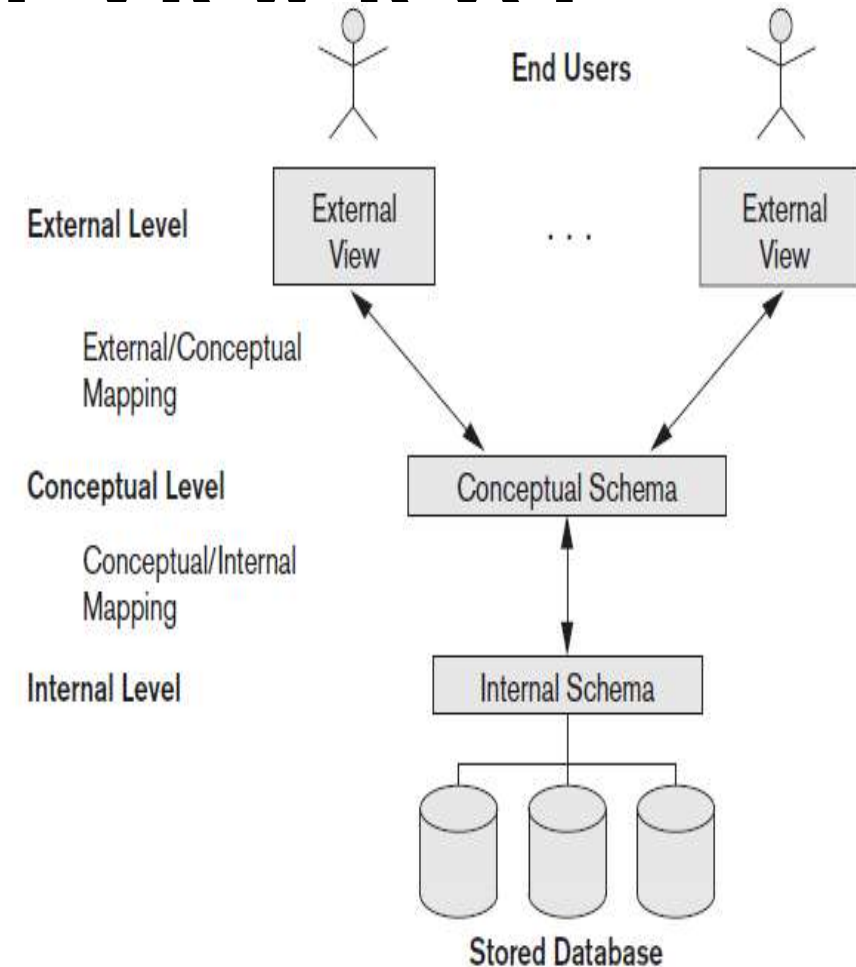


External Level/Schema or View level

- It is the users' view of the database.
- Describes only part of the database that a particular user group is interested in and hides the rest of the database from that group.
- It is closest to the end users.
- External level is also known as the view level.

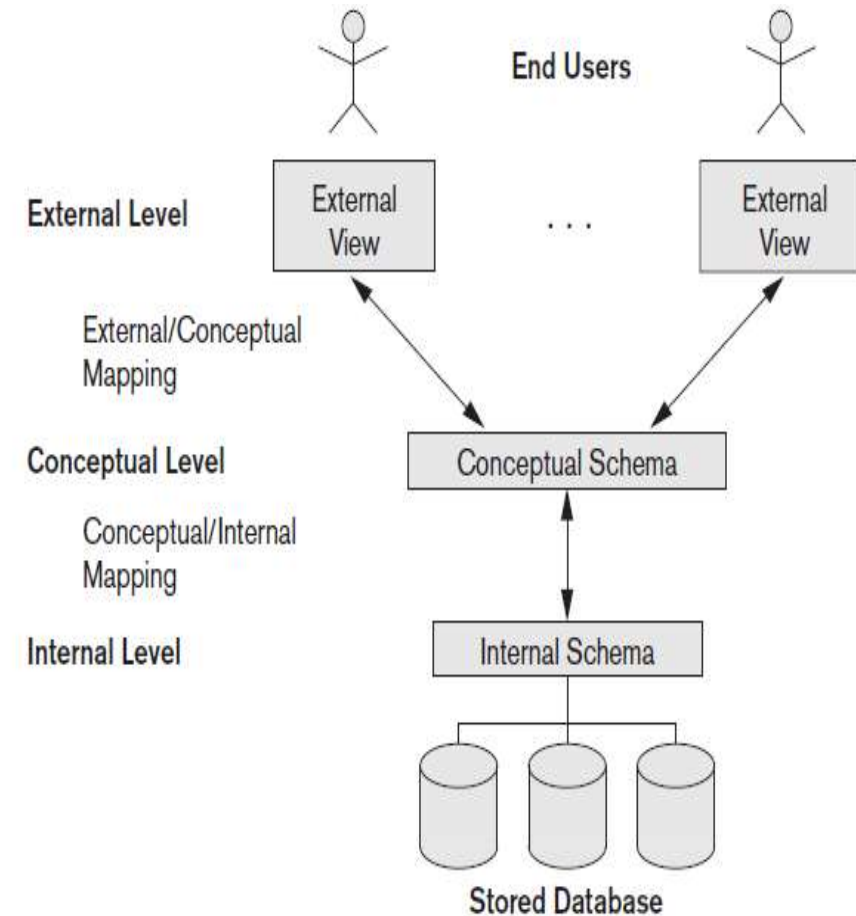
External Level/Schema or View level

- It deals with the way in which individual users view data.
- Individual users are given different views according to their requirements.
- Example: one user may view dates in the form (day, month, year), while another may view dates as (year, month, day).
- One may be interested in First Name other may be in First Name and Last Name.



Conceptual Level/Schema or Logical Level

- Describes what data are stored in the database and what relationships exist among those data
- Conceptual schema describe the structure and constraints for the whole database but not any storage considerations
- This level is used by the DBA who decides what information kept in the database
- Conceptual level is also known as the logical level.



Conceptual Level/Schema or Logical Level

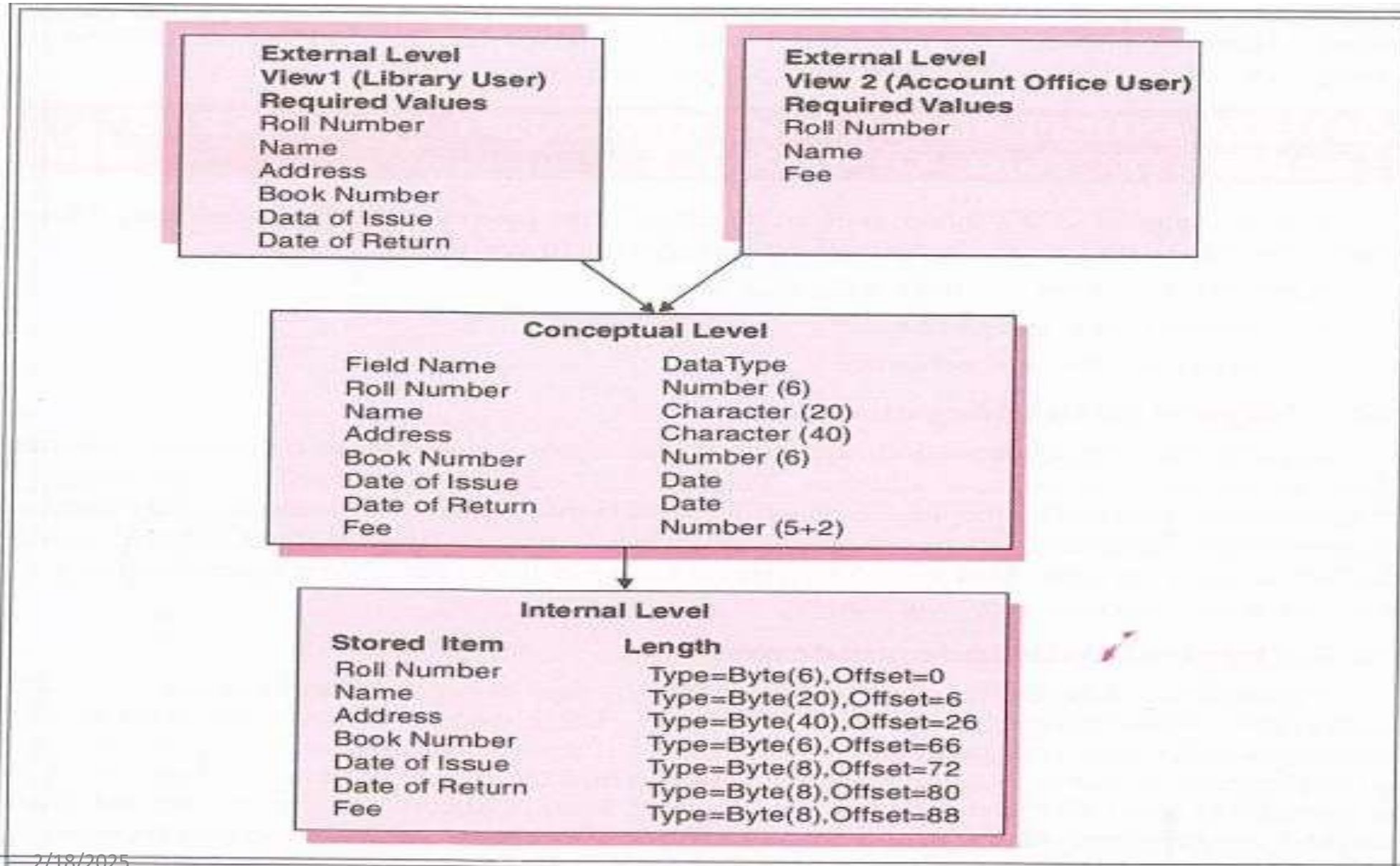
This level deals with

- All entities, their attributes, and their relationships
- The constraints on the data
- Security and integrity information

Internal Level/Schema or Physical level

- It is the physical representation of the database on the computer.
- This level describes how the data is stored in the database.
- It concerns the way the data are physically stored on the hardware.
- Storage space allocation for data and indexes;
- Record descriptions for storage (with stored sizes for data items) and Record placement;
- Data compression and data encryption techniques.

Example: University Management System



Data Independence

- Application Programs should be independent from the way the data is stored and accessed.
- Changing the database schema at one level without affecting the schema at the higher level is called as data independence.

Data Independence-Achievement of Layered Architecture of DBMS

There are two kinds of data independence:

- Logical data independence
- Physical data independence

Logical data independence

Logical data independence indicates that the conceptual schema can be changed without affecting the existing external schemas. The change would be absorbed by the mapping between the external and conceptual levels.

The conceptual schema changes may be:

- ✓ Adding a new record type
- ✓ Change in Constraint
- ✓ Deleting a record type
- ✓ Modifying data type and size

Physical data independence

Physical data independence indicates that the physical storage structures or devices could be changed without affecting conceptual schema. The change would be absorbed by the mapping between the conceptual and internal levels.

Changes in physical schema may

- ✓ Using new storage device
- ✓ Using different storage structure
- ✓ Switching from one access method to another
- ✓ Modifying indexes etc

Types of Users

Database users in DBMS can **access the database and retrieve the data** from the database using applications and interfaces provided by the **Database Management System (DBMS)**.

According to the tasks performed by the database users on the databases, we can categorize them into **seven categories** as follows:

- Database Administrators (DBA)
- Database Designers
- System Analysts
- Application Programmers / Back-End Developers
- Naive Users / Parametric Users
- Sophisticated Users
- Casual Users / Temporary User

Database Administrator(DBA)

- **Database Administrators (DBA) are the most important type of database users in DBMS. Database Administrator is an individual or a team of users who defines the database schema and takes charge of controlling various levels of the database within the organization.**
- **The function of DBA includes**
 - **Schema Definition**
 - **Storage Structure and access method definition**
 - **Schema and Physical organization modification**
 - **Granting of authorization for data access**
 - **Check on data integrity**
 - **Back up and Recovery**
 - **Routine maintenance.**

Database Administrator(DBA)

Examples of the database administrator's routine maintenance activities are:

- Periodically backing up the database onto remote servers, to prevent loss of data in case of disasters such as flooding.
- Ensuring that enough free disk space is available for normal operations, and upgrading disk space as required.
- Monitoring jobs running on the database and ensuring that performance is not degraded by very expensive tasks submitted by some users.

Database Designers

- As the name suggests, Database Designers are the users in DBMS who design and create the [structure of the database](#) including triggers, indexes, schemas, entity relationships, tables, constraints,, etc. which complete the database.
- Database designers try to gather information depending upon the requirements related to the database like the layout, looks, database functioning, costing, technologies to be used & implementation techniques, and finally, they design the final layout of the database for programmers to code its logic.
- Database Designers are the type of database users in DBMS who are responsible for implementing the overall design of the database.

System Analysts

- System Analysts are the type of database users in DBMS who analyze the requirements of Naive / Parametric End users. It is their responsibility to check whether all the requirements of end users are satisfied or not.
- Analyzing feasibility, economic, and technical aspects are some of the major responsibilities for a system analyst in DBMS. Sometimes, they are also responsible for the design, structure & functioning of the database.
- Application programmers+ System Analyst = Software Engineers

Application Programmer

- Application Programmers also known as Back-End Developers, are computer professional users who are responsible for developing the application programs (C, C++, Java, PHP, Python, etc.) or the user interface so that other users can use these applications to interact with the database.
- Application Programmers have deep knowledge of DBMS & databases and know everything in detail.
- They interact with the database using DML (Data Manipulation Language) queries to store data inside the database and when needed, they can also fetch the data from it.

Naive or Parametric Users

- Naive users also known as Parametric End users, don't have any knowledge of DBMS but still frequently use the database applications to get the desired results.
- With the help of the interface provided by the DBMS applications, Naive users mostly use the database to fill in or retrieve the information (view level of the database).
- Naive users don't need to be aware of the presence of the database system as they can interact with the database with the help of menu-driven application interface.
- For example, Railway ticket booking users in general are naive/parametric end users as they don't have much knowledge about DBMS and directly use railway booking applications to book their tickets.

Sophisticated Users

- Includes engineers, scientists, business analysts thoroughly familiar database system
- They are aware of presence of database.
- They interact with the system without writing programs, instead they form their requests in a database query language.
- These users are also known as SQL programmers as they can interact with the database directly using SQL queries using query processors. Using SQL queries, they can fetch the data from the database. They can also delete, update or insert new data into the database.

Casual Users

- Casual users also known as temporary users, are the type of database users in DBMS who frequently or occasionally use the database services.
- Casual/Temporary users have little knowledge about DBMS and each time they try to access the database, they require new information.
- For example: High-level management people are casual users who have little knowledge about DBMS and hence, they can access the database to either fill in new information or retrieve existing results.

Database Languages

1. DML(Data Manipulation Language)
2. DDL(Data Definition Language)
3. DCL(Data Control Language)
4. TCL(Transaction Control Language)

Data-Manipulation language (DML)

Enables users to access or manipulate data. The operations that can be performed include:-

Retrieval of information stored in the database.

Insertion of new information.

Deletion of information.

Modification of information stored in the database.

Types of data-manipulation language:

- **Procedural DML** specify what data are needed and how to get those data.
- **Declarative DMLs** (also referred to as nonprocedural DMLs) specify what data are needed without specifying how to get those data.

- QUERY:- A statement requesting the retrieval of information. The portion of DML that involves information retrieval is called query language.
- ✓ SELECT : retrieve the data from the database
- ✓ INSERT: insert into a table
- ✓ UPDATE: update the existing data within a table
- ✓ DELETE: Delete the records from a table

Data Definition Language (DDL)

It is used for defining the database schema

Example: create table *account* (
 account-number number(10),
 balance number(10,2));

DDL compiler generates and stored schema description in a *data dictionary or catalog*. *Data Dictionary is a special type of table consulted before any query or modification of the data.*

- CREATE- Create a database object like table, database
- ALTER- Alter existing table, column description
- DROP- Delete the existing object from the database
- Rename- Rename an existing database object

Data Control Language (DCL)

- ❑ DCL commands are used to grant and take back authority from any database user.
 - ❑ GRANT – Gives user's access privileges to the database
 - ❑ REVOKE- Withdraw the access privileges from the user given with the Grant command

Transaction Control Language (TCL)

- ❑ Transaction Control Language commands are used to manage transactions in the database.
- ❑ Used to manage the changes made by DML-statements.
- ❑ Examples of TCL commands
 - ✓ **COMMIT**: Permanently Save any update done into the database.
 - ✓ **SAVEPOINT**: Temporarily save any update so that you can later rollback to that point whenever necessary.
 - ✓ **Rollback**: Restores the database to last committed state in case of failure. It is also used with savepoint command to jump to a savepoint in a transaction.

Data Models

□ Data Models:

- A set of concepts to describe the **structure** of a database, the **operations** for manipulating these structures, and certain **constraints** that the database should obey.
- A collection of conceptual tools for describing
 - Data
 - Data Relationships
 - Data Semantics
 - Data Constraints
- A relatively simple representations, usually graphical, of complex real world data structures
- Provides necessary means to achieve abstraction

TYPE OF DATA MODELS

1) Object –Based or or Logical or High Level Models:

- a) Entity-Relationship Model
- b) Object-Oriented Model

2) Representational or Record-Based logical models:

- a) Relational Model
- b) Network Model
- c) Hierarchical Model

3) Physical Data Models

1) Object-Based Logical Models

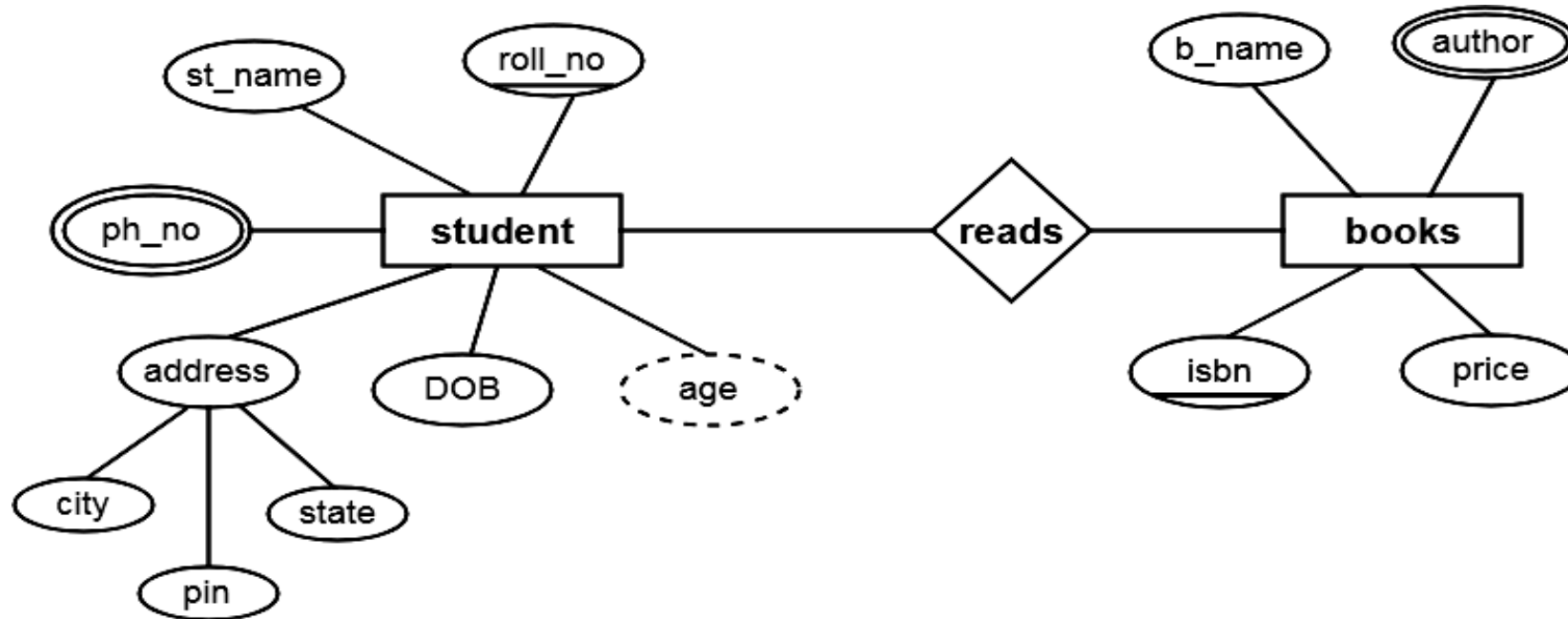
- i) Used in describing data at the logical & view level.
- ii) Allows constraints to be specified explicitly.

CATEGORIES:-

- a) Entity-relationship model
- b) Object-oriented model

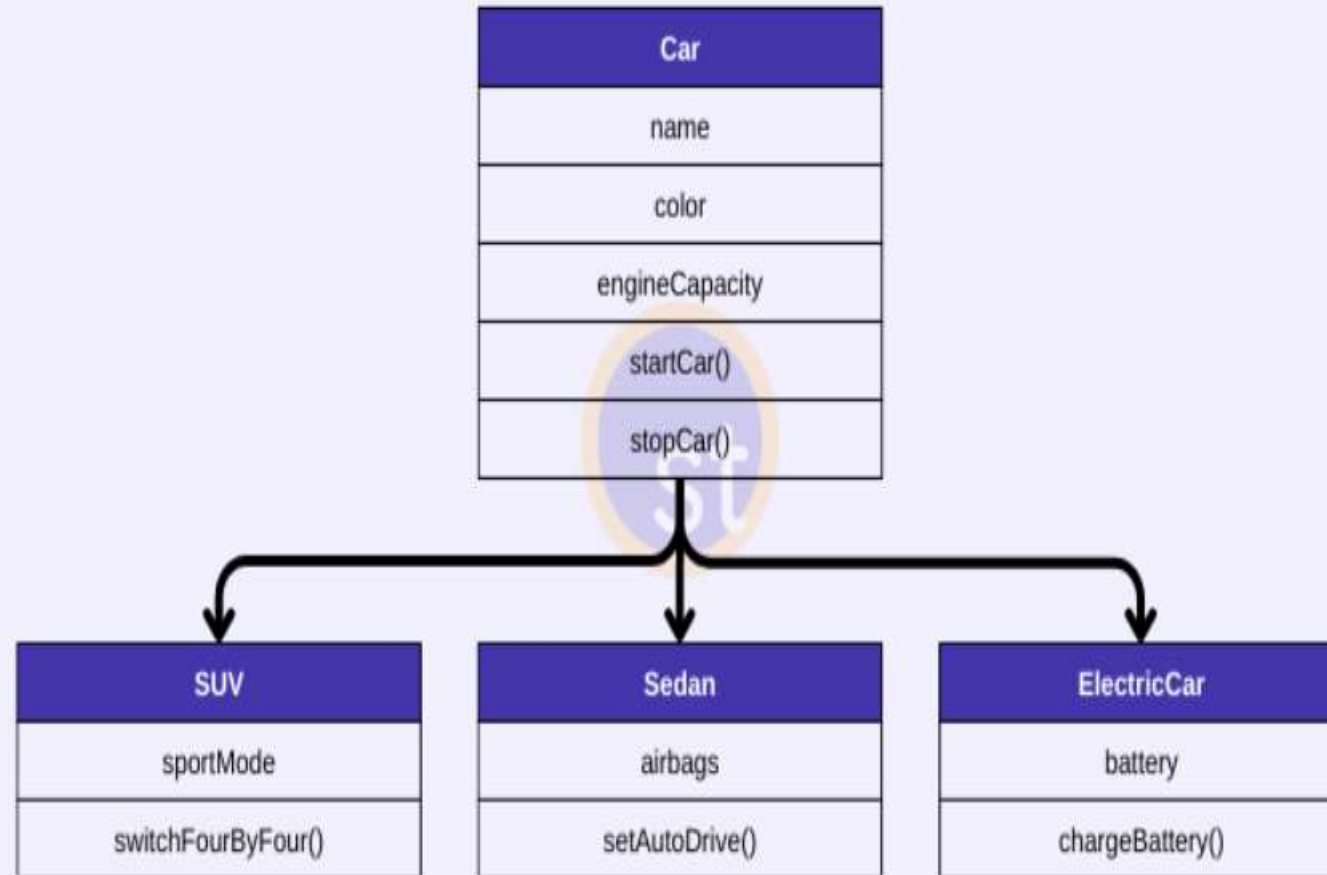
Entity-Relationship Model

- Models an enterprise as a collection of *entities* and *relationships*
 - **Entity**: a “thing” or “object” in the enterprise that is distinguishable from other objects of an organization
 - Described by a set of attributes or properties
 - **Relationship**: an association among several entities



Object-oriented Model

- In this model, data is stored in the form of objects.
- The behavior of the object-oriented database model is just like object-oriented programming.
- A very popular example of an Object Database management system or **ODBMS** is **MongoDB** which is also a NoSQL database.



Advantages of the Object-oriented Model

1. It can easily support complex data structures, with relationships.
2. It also supports features like Inheritance, Encapsulation, etc.

2) Record-Based Logical Models

Named so because DB is structured in **fixed-format records**.

Each record consists of fixed number of fields or attributes.

Each field is usually of a fixed length.

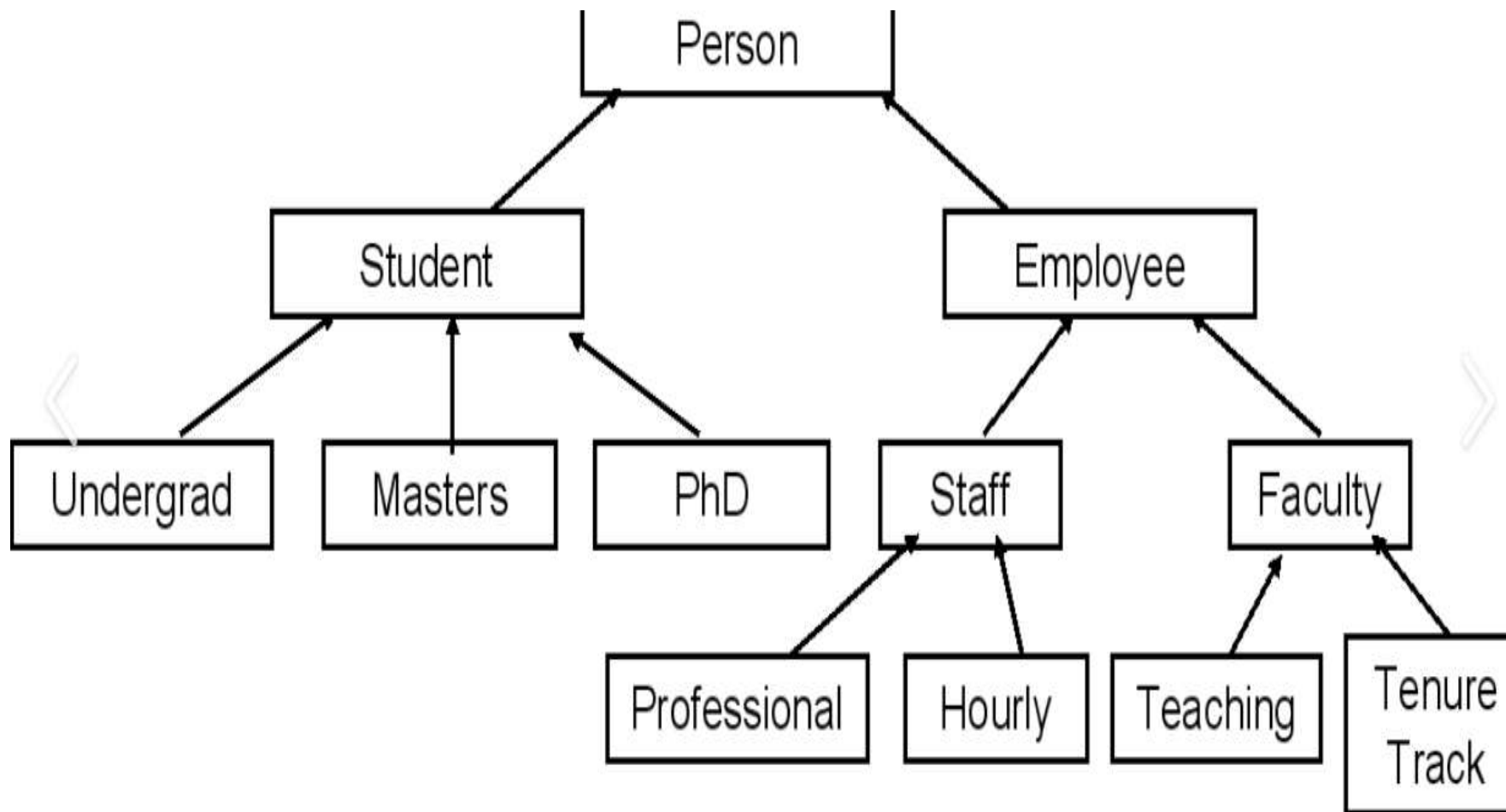
CATEGORIES:-

- a. Relational Model
- b. Network Model
- c. Hierarchical Model

Hierarchical model

- The hierarchical database model organizes data into a **tree-like structure**, with a **single root**, to which all the other data is linked.
- The hierarchy starts from the **Root** data, and expands like a tree, adding **child** nodes to the **parent** nodes.
- In this model, a child node will only have a **single parent node**.

- This model efficiently describes many real-world relationships like the *index of a book*, etc.
- Data is organized into a tree-like structure with a **one-to-many relationship** between two different types of data, for example, one **department** can have many **courses**, many **teachers**, and of course many **students**(like shown in the diagram below).



- Advantages/Disadvantages

1. Because it has one-to-many relationships between different types of data so it is easier and fast to fetch the data.
2. But the Hierarchical model is less flexible.
3. And it doesn't support many-to-many relationships.

Relational Model

- In this model, data is organized in two-dimensional **tables** and the relationship is maintained by storing a common field.
- This model was introduced by **E.F Codd** in 1970, and since then it has been the most widely used database model.
- The basic structure of data in the relational model is **tables**. All the information related to a particular type is stored in **rows** of that table.

- Hence, tables are also known as **relations** in the relational model.
- You can design tables, normalize them to reduce data redundancy, and use Structured Query language or SQL to access data from the tables.
- Some of the most popular databases are based on this database model. For example, **Oracle**, **MySQL**, etc.

student_id	name	age
1	Akon	17
2	Bkon	18
3	Ckon	17
4	Dkon	19

subject_id	name	teacher
1	Java	Mr. J
2	Python	Miss Py
3	JavaScript	Mrs. JS
4	C#	Mr. C



student_id	subject_id	marks
1	1	98
1	2	77
2	1	76
3	2	88

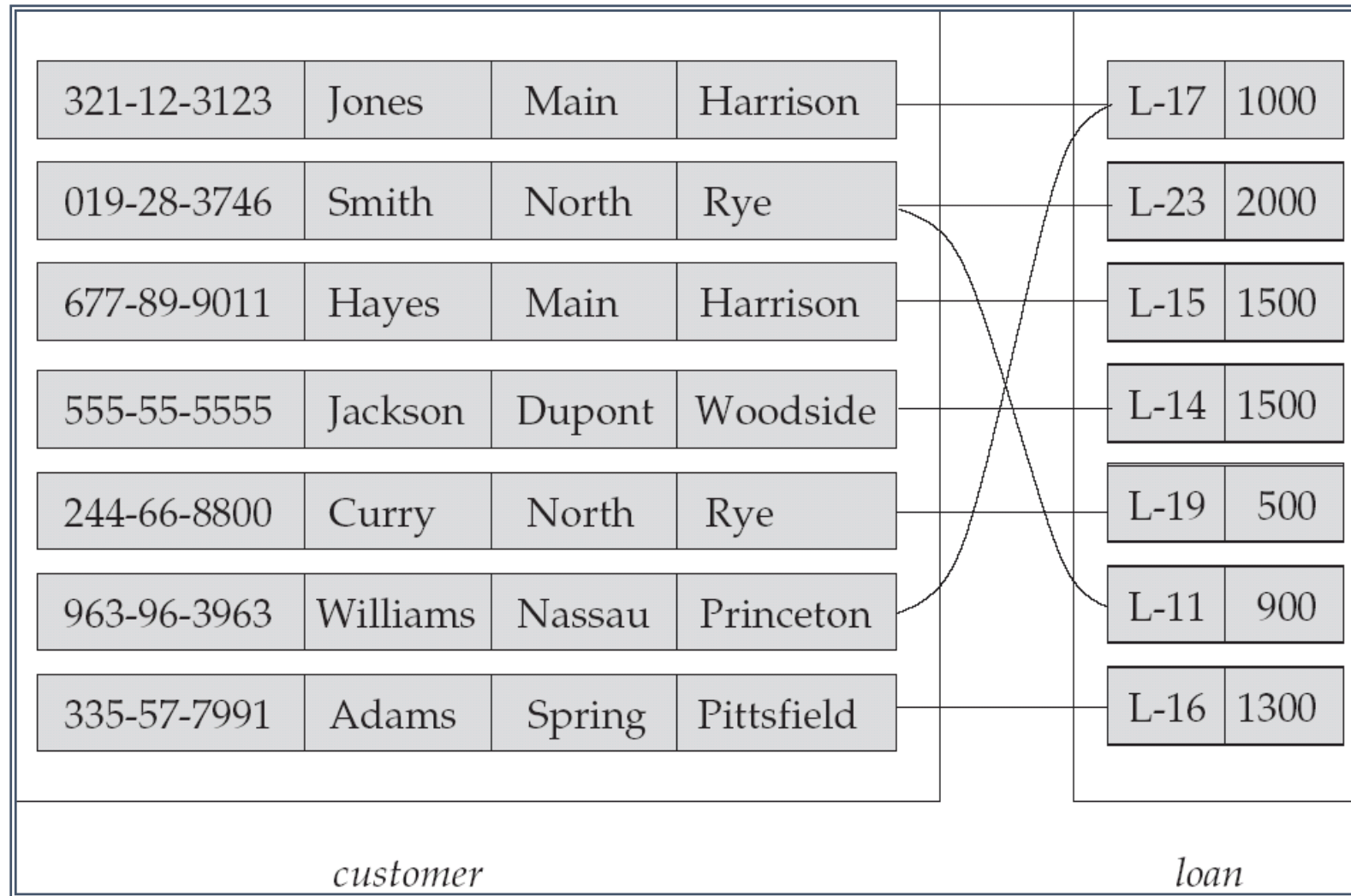
Advantages of the Relational Model

- 1.It's simple and easy to implement.
- 2.Popular database software is available for this database model.
- 3.It supports SQL using which you can easily query the data.

Network model

- The Network Model is an extension of the Hierarchical model.
- In this model, data is organized more like a graph, and allowed to have more than one parent node.
- In the network database model, data is **more related** as more relationships are established in this database model.
- Also, as the data is more related, hence **accessing the data is also easier and fast**.

- This database model uses many-to-many data relationships.
- This was the most widely used database model before Relational Model was introduced.
- The implementation of the Network model is complex, and it's very difficult to maintain it.
- The Network model is difficult to modify also.



- Advantages of the Network Model
 - 1.It supports complex relationships
 - 2.It allows more flexibility

ER Model

- ER model stands for an Entity-Relationship model. It is a high-level data model used for identifying the entities, their characteristics and relationships among various entities.
- In ER modeling, the database structure is portrayed as a diagram called an entity-relationship diagram.

For example, Suppose we design a school database. In this database, the student will be an entity with attributes like Address, Name, Stud_id, Age, etc. Teacher can be another entity with attributes like Name, Teacher_id, Salary, Dept. etc and there will be a relationship between them.

Terminologies used in ER model

Entity:

- An **entity** is an object that has some physical or logical existence in this real world.
Example: A person, A company, A plant etc..
- Entities have *attributes*
Example: A person has *name, address, age etc.*

Entity Type:

- An **entity type** is the collection of entities of similar attributes.
Example: Employee, Student, Department, Customer etc.

Entity Set:

- An **entity set** is a set of entities of the similar entity types.
Example: set of all employees, set of all students etc.

Terminologies continued....

Attribute:

An entity is represented by a set of attributes, that is descriptive properties possessed by all members of an entity set.

Domain: The set of permitted values/ possible values for each attribute is called its domain.

Types of Attributes:

- Simple and composite attributes.
- Single-valued and multi-valued attributes.
- Stored vs Derived attributes.
- NULL value attribute

Terminologies continued....

➤ Simple attribute:

An attribute which has single component i.e., it can not be further subdivided is called simple attribute. It has a single value for a particular entity.

Example: Regd_no, Account_no etc.

➤ Composite attribute:

An attribute which has multiple components i.e., it can be further subdivided is called simple attribute.

Example: Address, Name etc.

➤ Single-valued attribute:

An attribute which has single value for a single entity to which it belongs is called single-valued attribute.

Example: Roll no attribute for Student entity, Employee_ID attribute for Employee entity etc.

Terminologies continued....

➤ Multi-valued attribute:

An attribute which has multiple values for a single entity to which it belongs is called multi-valued attribute.

Example: Degree of a Person, Color of a Car etc.

➤ Derived attribute:

An attribute which can be derived from a stored attribute is called derived attribute and from which it is derived is called stored attribute.

Example: Age can be derived from DoB attribute

➤ NULL valued attribute:

In some cases the particular entity may not have a value for a particular attribute then it is NULL.

Ex: Phone_no(Home) and Phone_no(Office)

Terminologies continued....

Relationship types: A relationship type is the association among several entities.

Degree of a Relationship: The no of entity types participating in the relationship is called the degree of a relationship.

Example: If one entity type is participating then it is a recursive relationship and degree of relationship=1

If two entity types are participating then it is a binary relationship and degree of relationship=2 and so on.

If three entity types are participating then it is called a ternary relationship and degree=3.

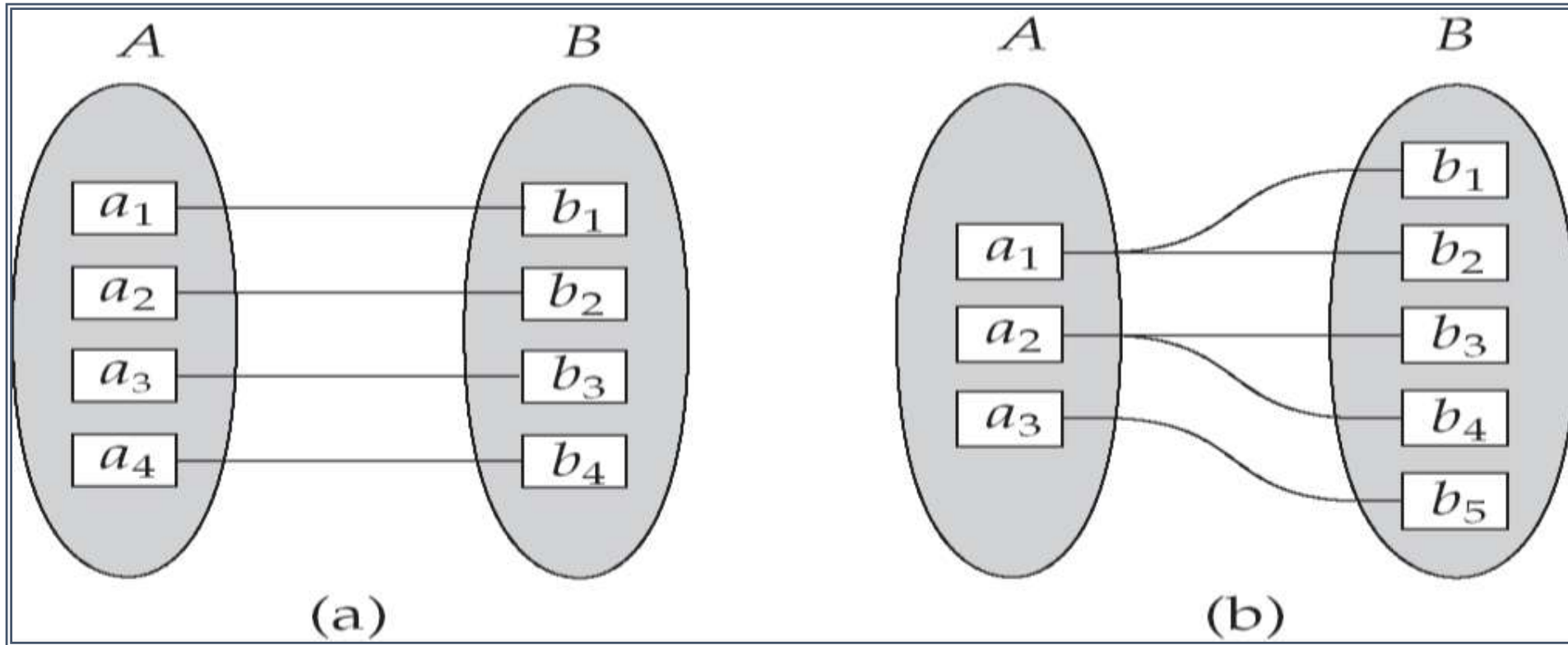
Note: Relationship types can have attributes

example: Manages relationship type has mgr_date attribute

Mapping Cardinality Constraints

- It tells us the number of entities to which another entity can be associated via a relationship set.
- The mapping cardinality must be one of the following types:
 - One to one
 - One to many
 - Many to one
 - Many to many

Mapping Cardinalities



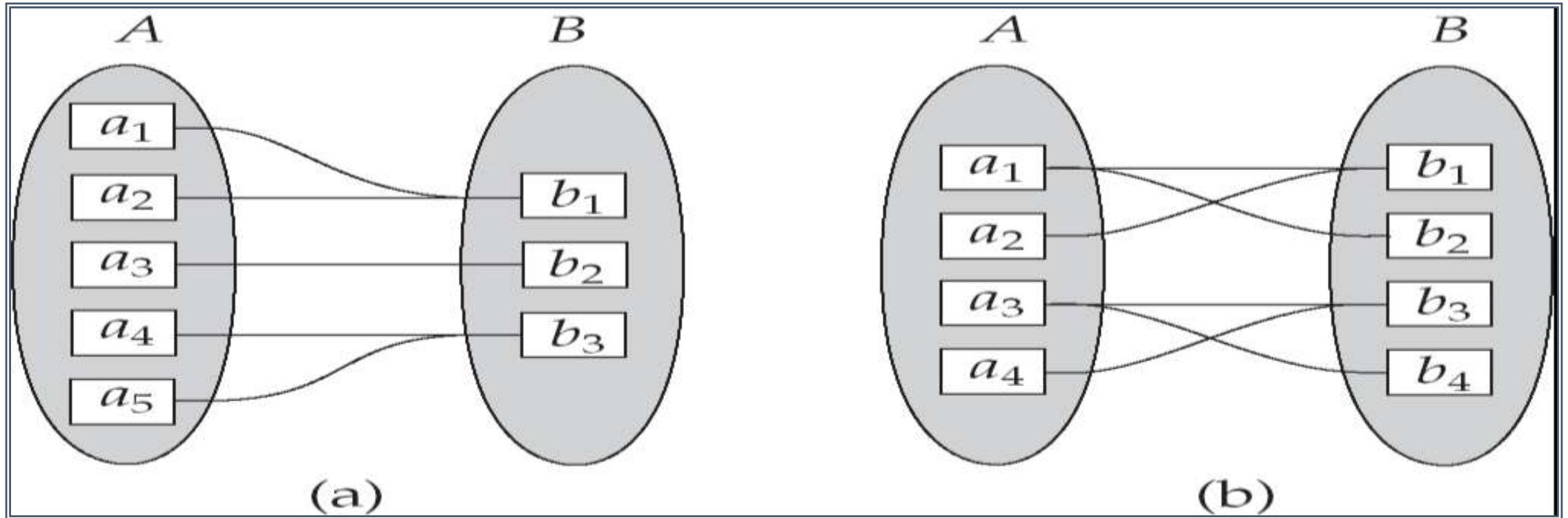
One to one

An entity in set A can be associated to one and only one entity in set B.

One to many

An entity in set A can be associated to any no of entities in set B but an entity from B can be associated with one entity from set A

Mapping Cardinalities



Many to one

An entity in set B can be associated to any no of entities in set A.

Many to many

An entity in set A can be associated to any no of entities in set B and vice versa.

Participation constraint

It specifies how each entity is involved in the relationship.

There are two types of Participation constraint:

- Total participation
- Partial participation

Total participation constraint

- It specifies that **each entity present in the entity set must mandatorily participate in at least one relationship instance of that relationship set,**
- for this reason, it is also called as mandatory participation
- It is **represented using a double line** between the entity set and relationship set

Partial participation

- It specifies that **each entity in the entity set may or may not participate in the relationship instance of the relationship set,** is also called as optional participation
- It is represented using a **single line between the entity set and relationship set** in the ER diagram

Weak and Strong Entity Types

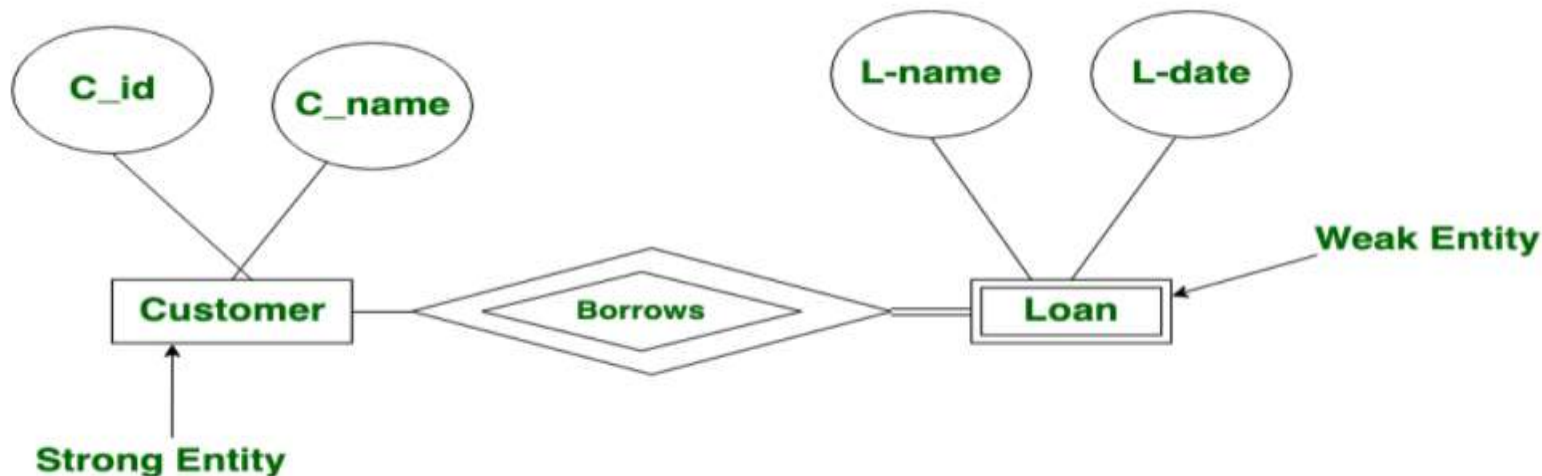
Strong Entity:

- An entity is called a strong entity, if it possesses a primary key.

Example: Student

Weak Entity:

- An entity that does not have sufficient attributes to form a primary key that entity is called weak entity.
- Example: Loan
- The weak entity has existence dependency on the strong entity to which it belongs. So, the primary key of the weak entity can be formed by combining the primary key of the owner entity and the partial key of the weak entity



Key Concepts

What is a Key?

- Keys play an important role while creating a table.
- Keys are used to identify a row(tuple) in a relation(table) uniquely..
- They may have a single attribute.: *Example*: Emp_ID, or a group of attributes:
Example: (Name, DOB, Address)
- Mostly used for establishing the relationships among the various tables.

Types of Keys:

- Super key
- Candidate key
- Primary key
- Alternate key
- Foreign key

Super Key

- A super key of an entity set is a set of one or more attributes whose values uniquely determine each entity.
 - Example in the entity set employee, employee number and employee name is a super key
 - employee name alone is not a super key as two employees could have same name.
 - By default set of all attributes of an entity type is the super key of that entity type.
 - A super key may contain extraneous attributes, and we are often interested in the smallest super key.
-
- Example: In Employee Table
 - Emp_ID,
 - (Emp_ID, Name, DOJ),
 - (Email_ID, DOB, Department) ... etc
 - *Real_Life Examples:*
 - PAN No + Name
 - Employee ID + Name
 - Aadhar No + DOB + Name

Candidate Key

- It is the minimal Super Key.
- Candidate keys should always be unique, can have NULL values.
- Have a single attribute (*Example*: Emp_ID), or a group of attributes. (*Example*: (Name, DOB,Address))
- A Table may have one or more Candidate Keys.
 - Example:
 - (Emp_ID),
 - (Email_ID),
 - (Name, DOJ),
 - (Name, Department),
 - (DOJ, Department), (Name, Department) ... etc. all are the Candidate keys of the Employee Table.
- Every Candidate key is a Super Key, vice-versa is not possible (as Super key is the addition of 0,1 or more attributes to a Candidate key)
 - Emp_ID – Candidate Key, Super Key
 - (Emp_ID, Name) – Super Key, Not Candidate Key
- **Real-Life Examples:**
 - Aadhar No
 - PAN No
 - Account No
 - Voter ID etc.

Primary Key:

- The best suitable Candidate key acts as the Primary key of the Table. (*Example: Emp_ID*)
- It is chosen by the D.B.A. (Database Administrator).
- A Table can have a single Primary key (If Emp_ID is Primary key, then other Candidate keys can't be the Primary key of the same table).
- The primary key should always be unique and non-null key, can't have NULL values.
- *Every Primary key is the candidate key*
- *A Candidate key may or may not be a Primary Key.*

Example:

(Emp_ID) – Candidate Key, Primary Key

(Email_ID) – Not Primary Key, Candidate Key

(Name, DOB) – Candidate Key, Not Primary Key

Alternate Key:

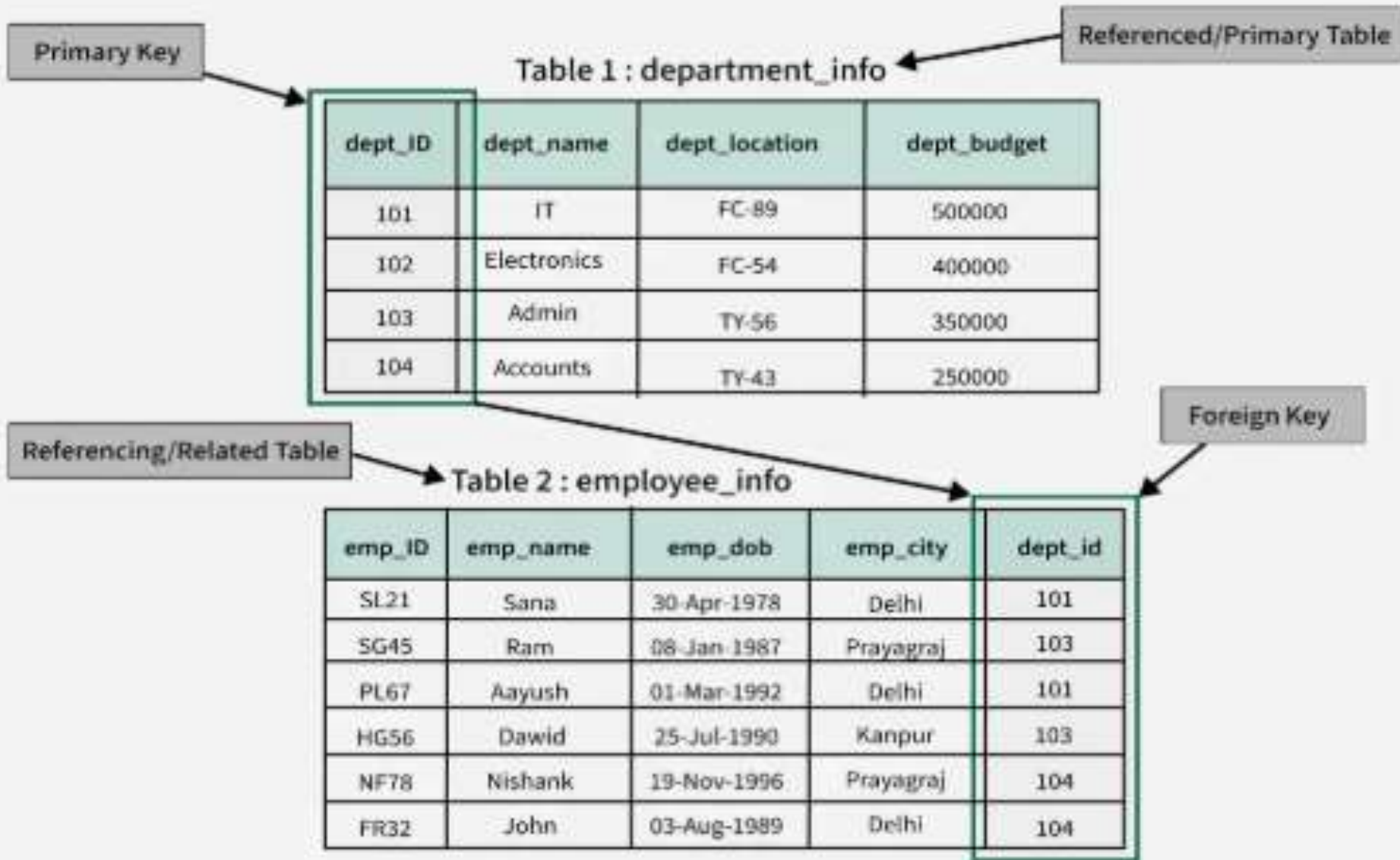
- The Keys other than Primary Key are called Alternate Keys (If Emp_ID is Primary key, then other Candidate keys are Called the Alternate Keys).

Example: (Email_ID), (DOB, Name), (DOB, DOJ, Department), (DOJ, Name), etc all are Alternate Keys.

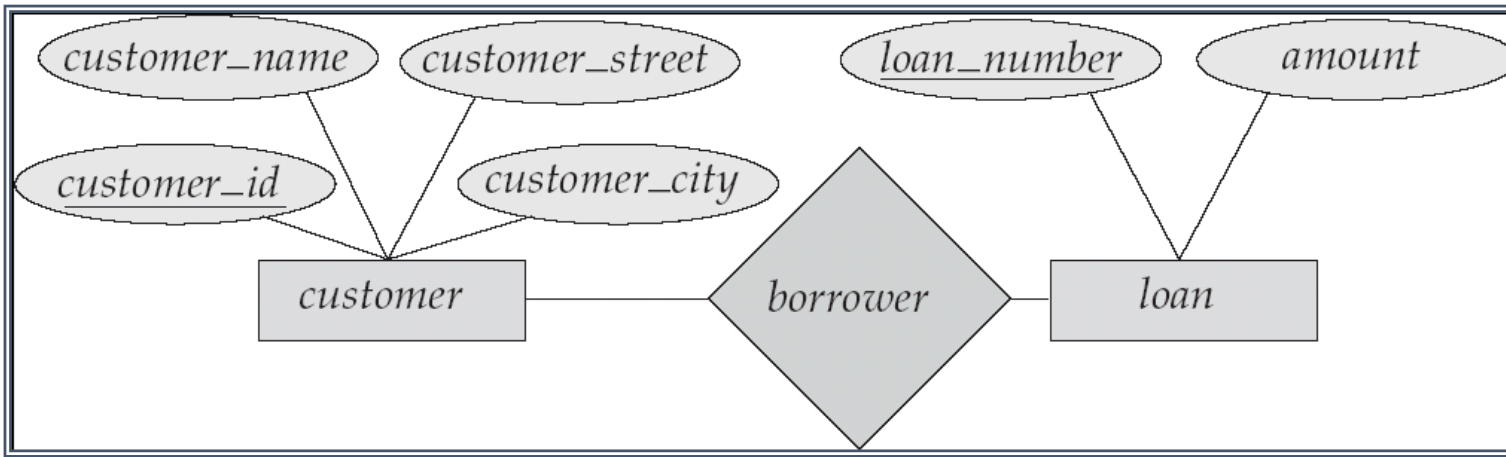
- Sometimes Alternate key is required for the indexing, for better and faster searching.

Foreign Key:

- It helps us in establishing relationships with other tables.
- Foreign Key refers to the Primary Key of another table (Example: dept_id of Employee table refers to dept_id which is the Primary Key of the of Department table.
- A foreign key is a field (or collection of fields) in one table that uniquely identifies a row of another table.
- The table containing the foreign key is called the child table, and the table containing the candidate key is called the referenced or parent table.



E-R Diagram Notations



- Rectangles represent entity sets.
- Diamonds represent relationship sets.
- Lines link attributes to entity sets and entity sets to relationship sets.
- Ellipses represent attributes
 - Double ellipses represent multivalued attributes.
 - Dashed ellipses denote derived attributes.
- Underline inside ellipse indicates primary key attributes.

E-R Diagram Notations

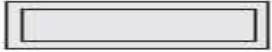
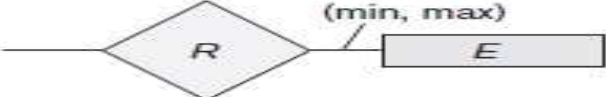
Symbol	Meaning
	Entity
	Weak Entity
	Relationship
	Identifying Relationship
	Attribute
	Key Attribute
	Multivalued Attribute
	Composite Attribute
	Derived Attribute
	Total Participation of E_2 in R
	Cardinality Ratio 1: N for $E_1:E_2$ in R
	Structural Constraint (min, max) on Participation of E in R

Figure 7.14
Summary of the notation
for ER diagrams.

Company DB

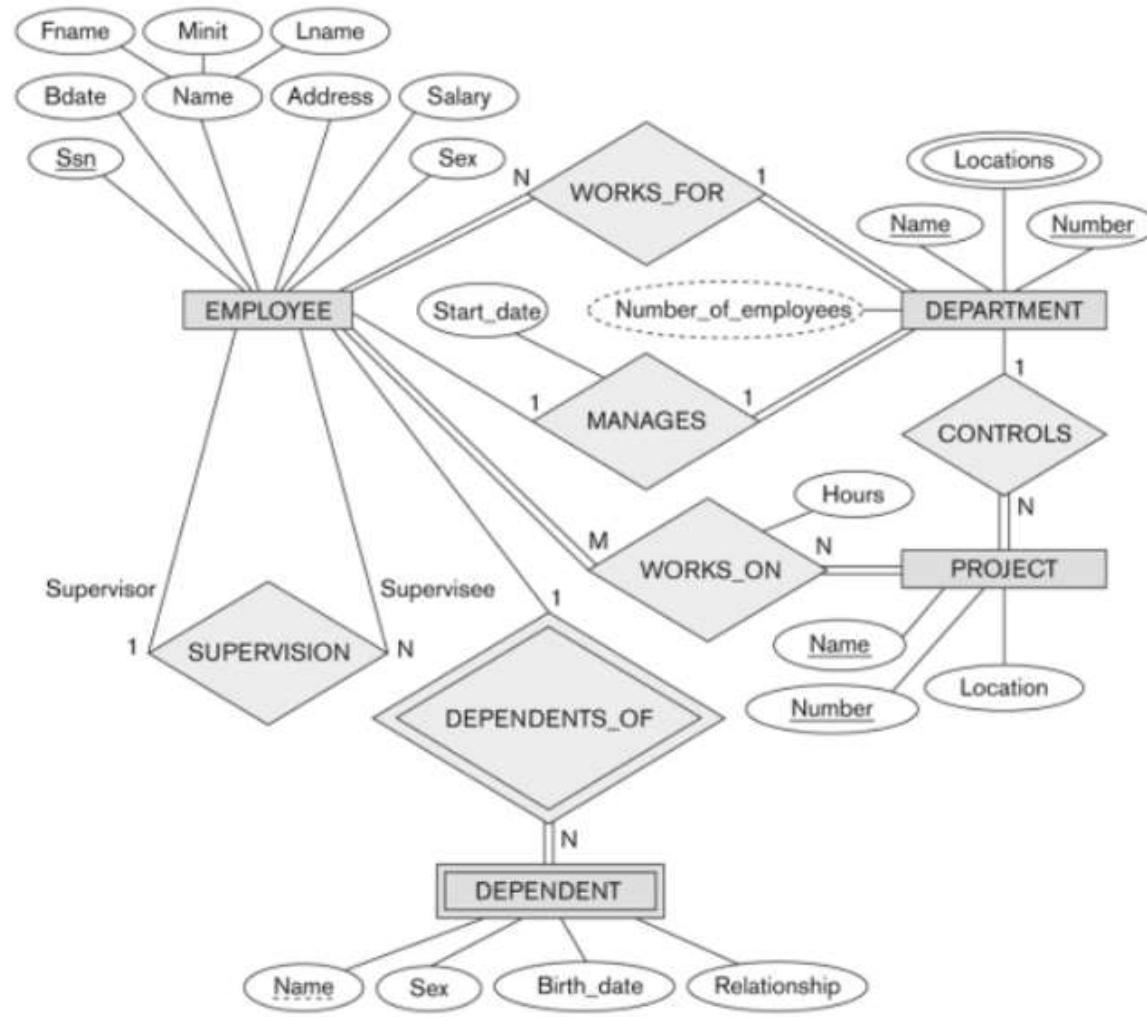
Requirements of the Company

- ✓ The company is organized into DEPARTMENTS. Each department has a name, number, location and an employee who *manages* the department. We keep track of the start date of the department manager.
- ✓ Each department *controls* a number of PROJECTS. Each project has a name, number and is located at a single location.

- ✓ We store each EMPLOYEE's social security number, address, salary, name, and birth-date.
- ✓ Each employee *works for* one department but may *work on* several projects.
- ✓ We keep track of the number of hours per week that an employee currently works on each project.
- ✓ We also keep track of the *direct supervisor* of each employee.
- ✓ Each employee may *have* a number of DEPENDENTS. For each dependent, we keep track of their name, address, birth-date.

- ✓ The company is organized into **DEPARTMENTS**. Each department has a name, number, location and an **employee** who *manages* the department. We keep track of the start date of the department manager.
- ✓ Each department *controls* a number **of PROJECTs**. Each project has a name, number and is located at a single location.
- ✓ We store each EMPLOYEE's social security number, address, salary, name, and birth-date.
- ✓ Each employee *works for* one department but may *work on* several projects.
- ✓ We keep track of the number of hours per week that an employee currently works on each project.
- ✓ We also keep track of the *direct supervisor* of each employee.
- ✓ Each employee may *have* a number of **DEPENDENTs**. For each dependent, we keep track of their name, address, birth-date.

E-R Diagram of Company Database



EXTENDED ENTITY-RELATIONSHIP MODEL (EER) MODEL

EER MODEL

- EER stands for **Enhanced ER** or **Extended ER**
- EER Model Concepts
 - Includes all modeling concepts of basic ER
 - Additional concepts:
 - **Generalization**
 - **Specialization**
 - **attribute and relationship inheritance**
 - **Aggregation**
 - These are fundamental to conceptual modeling
- The additional EER concepts are used to model applications more completely and more accurately
 - EER includes some **object-oriented concepts**, such as inheritance

Subclasses and Super-classes

- An entity type may have additional meaningful **sub groupings** of its entities
 - Example: EMPLOYEE may be further grouped into:
 - SECRETARY, ENGINEER, TECHNICIAN, ...
 - Based on the EMPLOYEE's Job
 - MANAGER
 - EMPLOYEES who are managers
 - SALARIED_EMPLOYEE, HOURLY_EMPLOYEE
 - Based on the EMPLOYEE's method of pay
- EER diagrams extend ER diagrams to represent these additional **sub groupings**, called *subclasses or subtypes*

Subclasses and Super-classes

- Each of these sub groupings is a subset of EMPLOYEE entities
- Each is called a subclass of EMPLOYEE
- EMPLOYEE is the **superclass** for each of these subclasses
- These are called **superclass/subclass** relationships:
 - EMPLOYEE/SECRETARY
 - EMPLOYEE/TECHNICIAN
 - EMPLOYEE/MANAGER
 - ...

Subclasses and Super-classes

- These are also called IS-A relationships
 - SECRETARY IS-A EMPLOYEE, TECHNICIAN IS-A EMPLOYEE,
- Note: An entity that is member of a subclass represents the same real-world entity as some member of the superclass:
 - The subclass member is the same entity in a *distinct specific role*
 - An entity cannot exist in the database merely by being a member of a subclass; it must also be a member of the superclass
 - A member of the **superclass** can be **optionally** included as a member of any number of its subclasses

Generalization and Specialization

- The EER Model has the power of **expressing entities** in a **conceptual hierarchical manner**.
- As the hierarchy goes up, it generalizes the view of entities, as we go deep, it gives the detail of every entity included.
- Going up in this structure is called generalization, where entities are clubbed together to represent a more generalized view
- The entity shall be a student and further the student is a person. The reverse is called specialization where a person is a student and the student is XYZ.

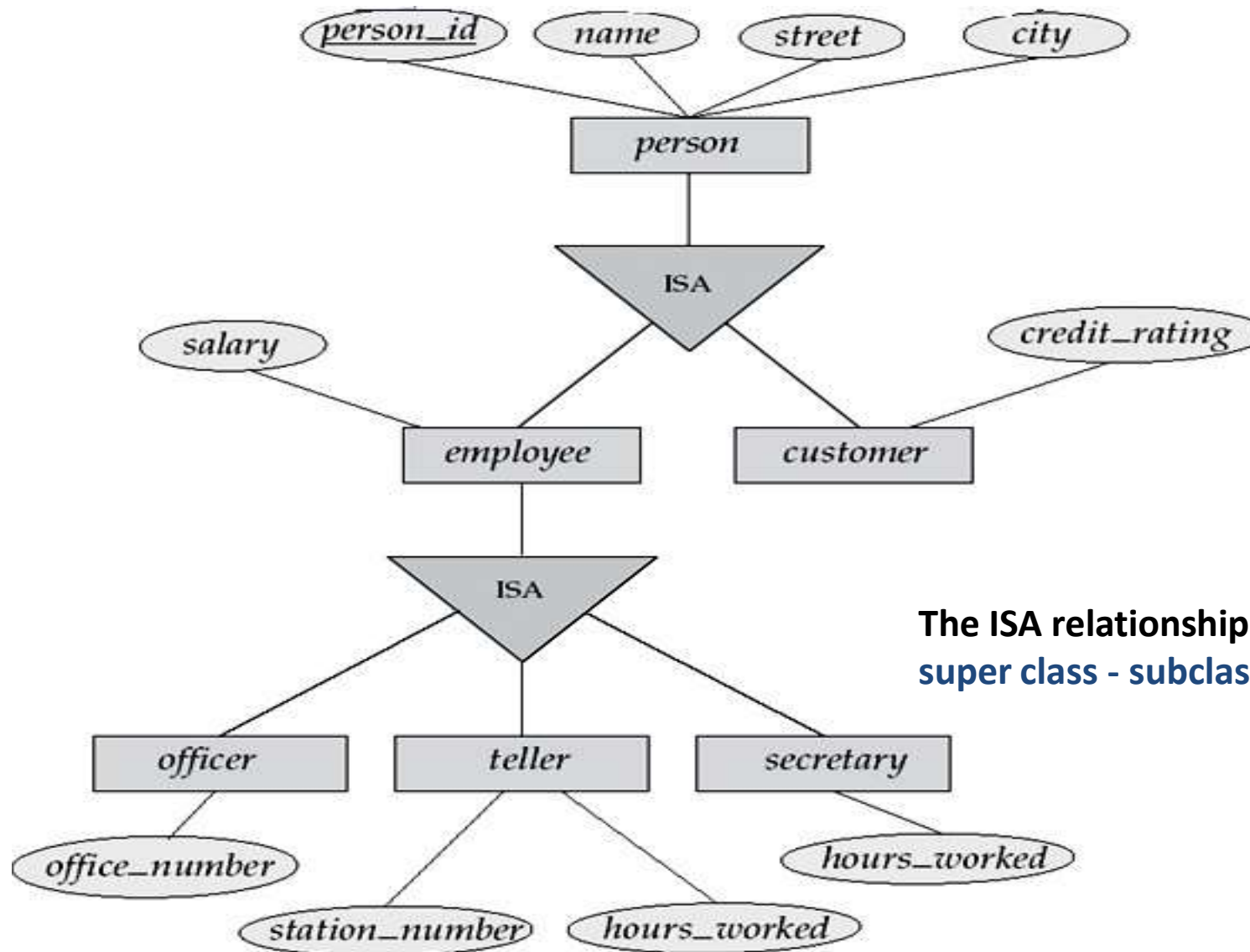
Generalization

- Several classes or entity sets with common features are generalized into a super class . original classes become its subclasses.
- **A bottom-up design process** – combine a number of entity sets that share the same features into a **higher-level entity set**.
- In generalization, a **number of entity sets** are brought together into **one generalized entity set** based on their **similar characteristics**. For example, pigeon, house sparrow, crow and dove can all be generalized as Birds.

Specialization

- Specialization is the reverse process of generalization.
- Top down design process (sub groupings within an entity set that are **distinctive** from other entities in the set).
- In specialization, a group of entities is **divided** into **sub-groups** based on their characteristics.
- The set of subclasses is based upon some **distinguishing characteristics** of the entities in the superclass.
- **In a school database, persons can be specialized as teacher, student, or a staff, based on what role they play in school as entities.**
- Specialization and generalization are simple **inversions** of each other; they are represented in an E-R diagram in the same way. The terms specialization and generalization are used **interchangeably**.

Example of Generalization / Specialization

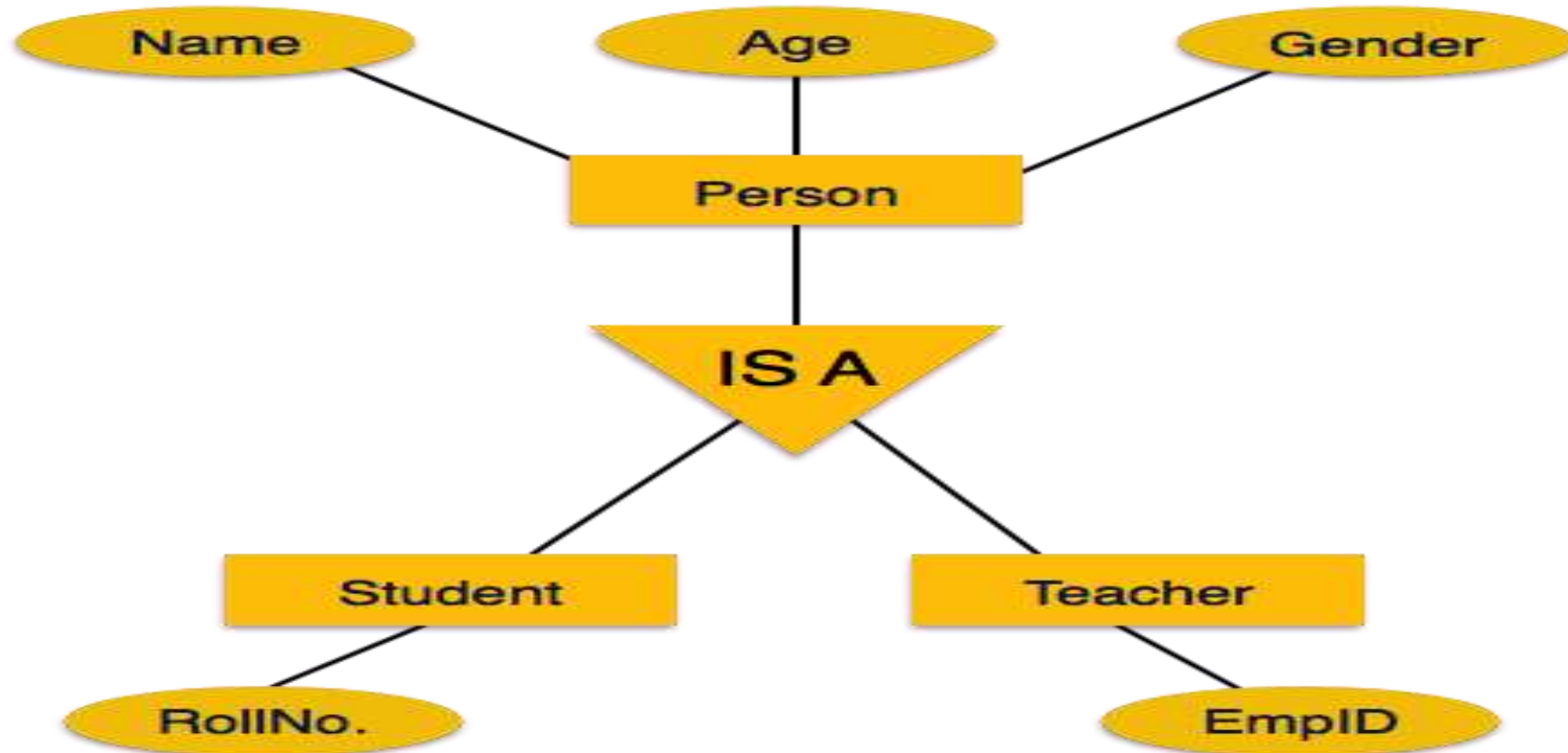


The ISA relationship also referred to as **super class - subclass** relationship

Inheritance

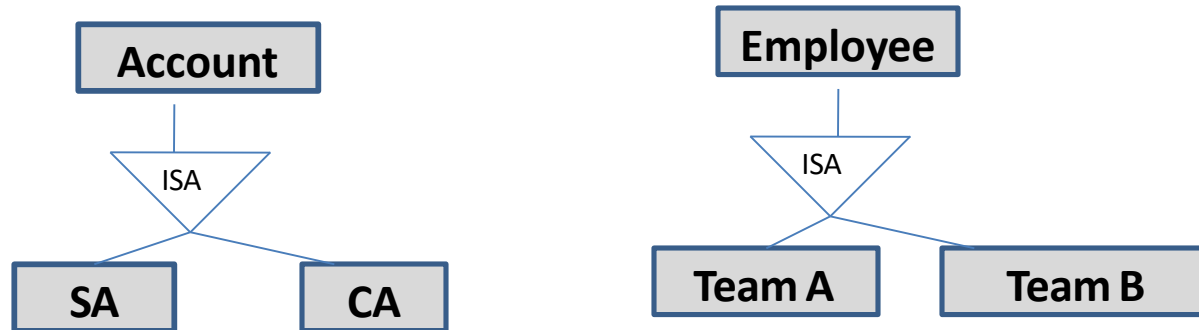
- All the above features of EER model are used in order to create **classes and objects** in **object oriented programming**.
- The details of entities are generally hidden from the user ; this process is known as **abstraction**
- **Inheritance** is an important feature of generalization and specialization. It allows **lower-level entities to inherit the attributes and relationship** of higher level entities .
- For example the attributes of a person superclass such as name, age and gender can be inherited by the lower level entities such as Student or Teacher.

Inheritance



Design Constraint On Generalization/Specialization

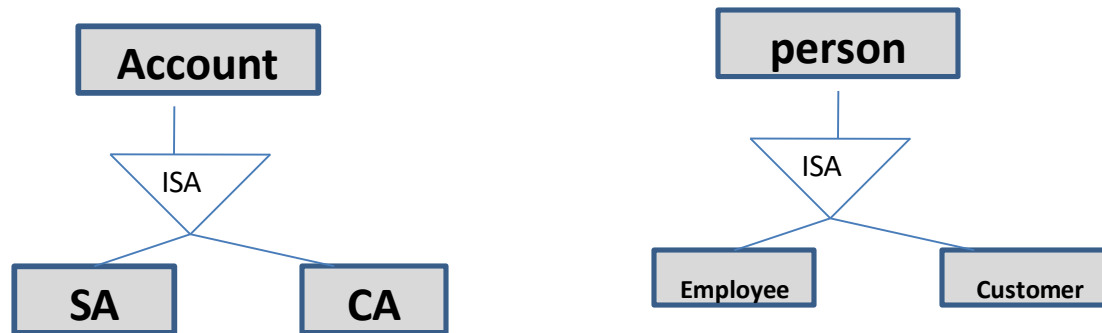
- Constraint which determines which entities can be members of a given lower-level entity set.
 - **condition-defined/ attribute-defined:** Membership evaluated on the basis of whether or not an entity satisfies an explicit condition or predicate.
 - **user-defined:** Here, the DBA assigns a particular entity into a lower level entity set.



- Constraint which determines whether or not entities may **belong to more than one lower-level entity set** within a single generalization.
 - **Disjoint:** A entity belongs to no more than one lower level entity set within a single generalization
 - **Overlapping:** Some entity may belongs to more than one lower entity set within a single generalization.

Design Constraint On Generalization/Specialization

- Constraint which determines whether or not entities may belong to more than one lower-level entity set within a single generalization.
 - **Disjoint:** A entity belongs to no more than one lower entity set within a single generalization
 - **Overlapping:** Some entity may belongs to more than one lower entity set within a single generalization.

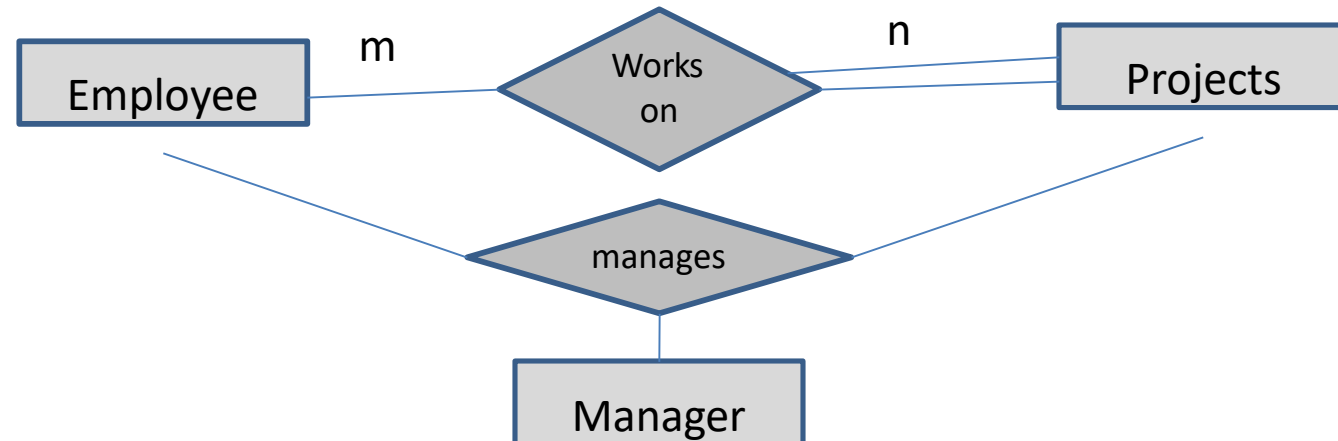


Design Constraints on a Specialization/Generalization (Cont.)

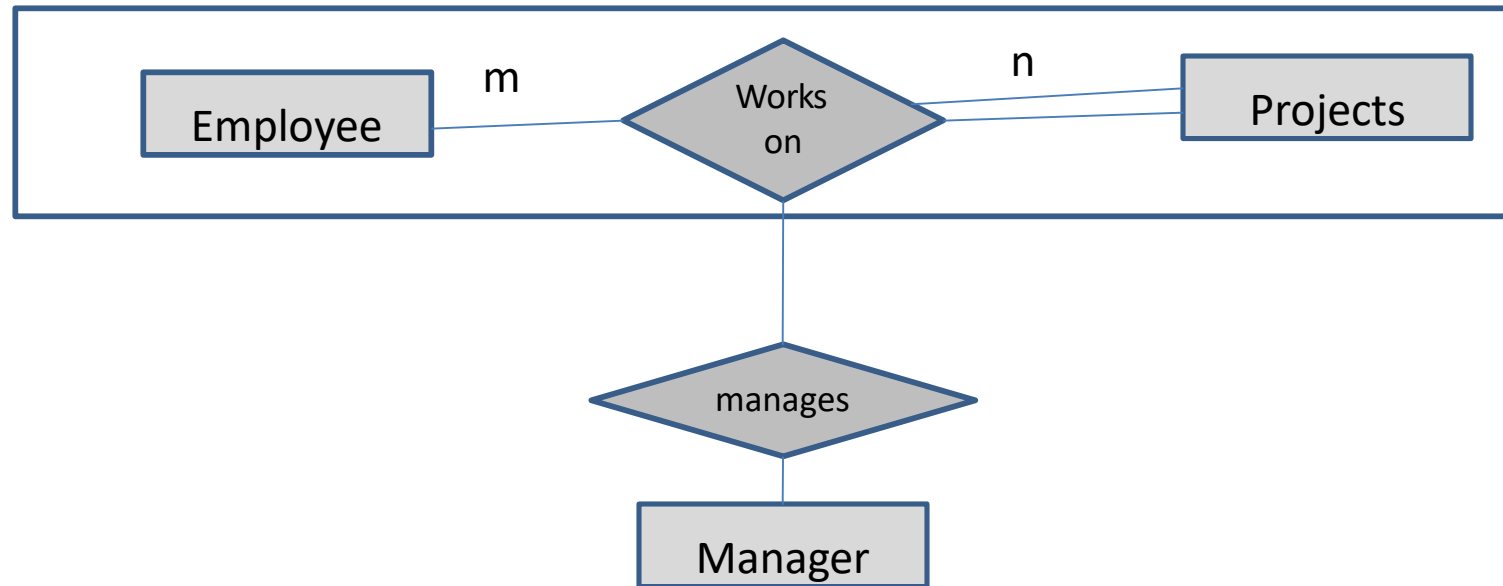
- Completeness constraint -- specifies whether or not an entity in the higher-level entity set must belong to at least one of the lower-level entity sets within a generalization.
 - total : Each higher level entity must belong to one of the lower-level entity sets
 - partial: Some higher level entity may not belong to one of the lower-level entity sets
- Hence, we have four types of specialization or generalization :
 - Disjoint, total
 - Disjoint, partial
 - Overlapping, total
 - Overlapping, partial

Aggregation

- In aggregation, the relation between two entities is treated as a single entity. Relationship with its corresponding entities is aggregated into a **higher level entity**.
- It is an abstraction through which relationship are treated as higher level entities where it is convenient to express relationship among relationship.



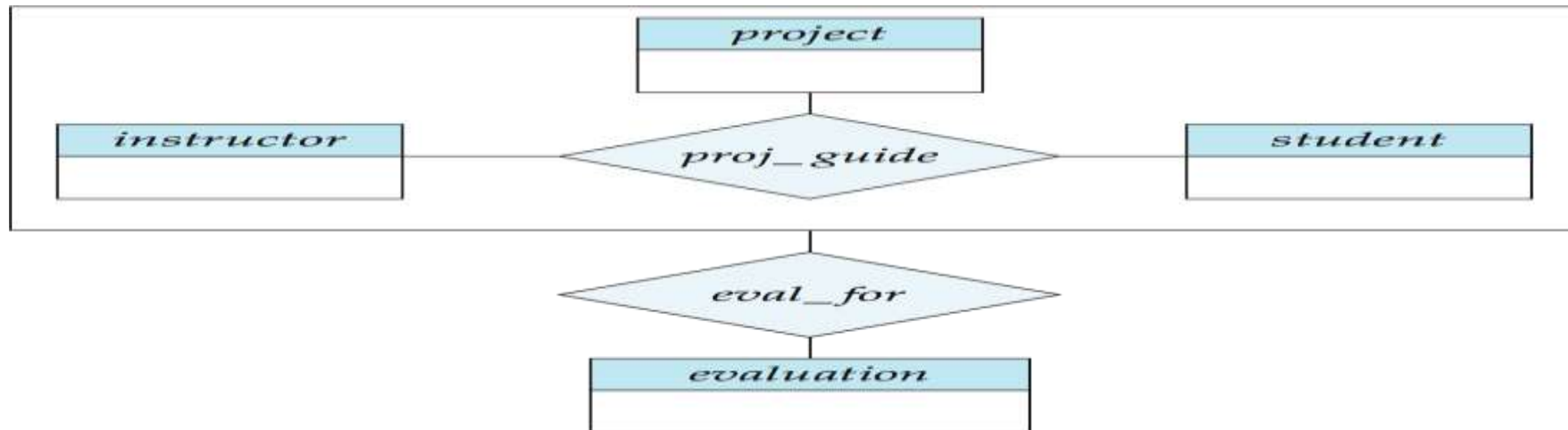
Aggregation



Whenever we need a relation between entity and another relationship there we require use aggregation.

Aggregation

- Eliminate this redundancy via *aggregation*
 - Treat relationship as an abstract entity
 - Allows relationships between relationships
 - Abstraction of relationship into new entity
- **Example**
 - A student is guided by a particular instructor on a particular project
 - A student, instructor, project combination may have an associated evaluation



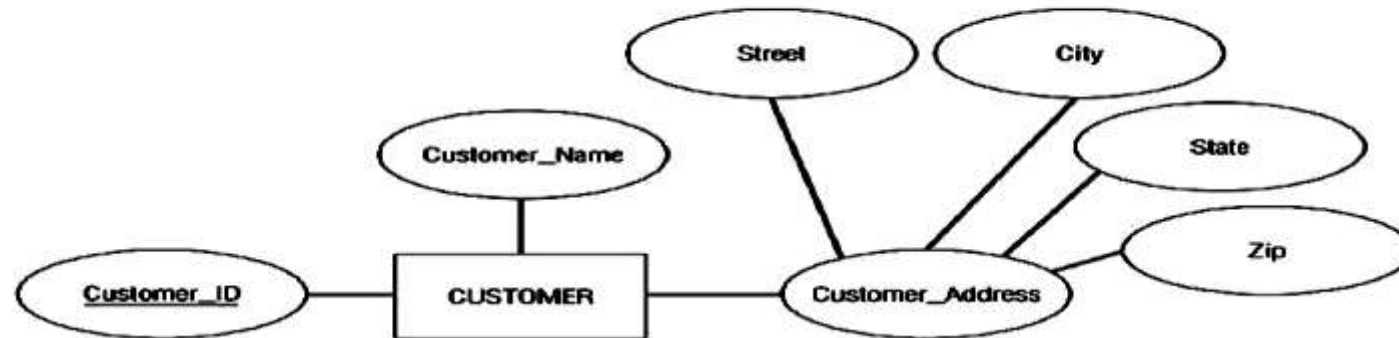
CONVERTING ER-MODEL TO RELATIONAL MODEL

1-Mapping of Strong Entity Set

- A strong entity set reduces to a relational schema schema with the same attributes.
 - Composite Attributes: Use only their
 - simple, component of composite attributes

Mapping a Composite Attribute

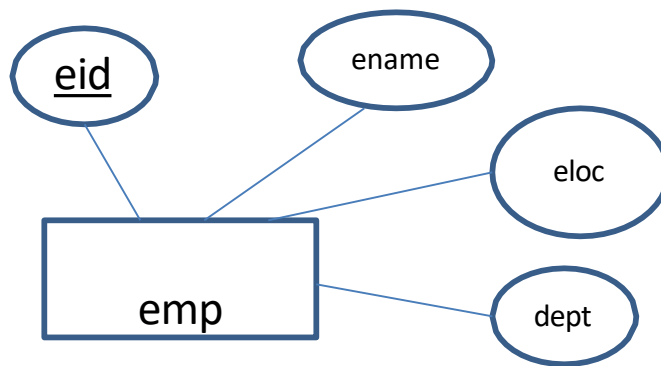
(a) **CUSTOMER** entity type with composite attribute



(b) **CUSTOMER** table with address detail

CUSTOMER					
<u>Customer_ID</u>	Customer_Name	Street	City	State	Zip

- Convert the following ER diagram into relational schema



Relational schema

Employee

<u>eid</u>	Ename	Eloc	dept
------------	-------	------	------

1-Mapping of Strong Entity Set

- Multivalued Attribute:
 - For each multi-valued attribute **A** in strong entity set **E** creates a separate relation or table **R** with **A** as an attribute and **primary key of E** as **foreign key in R**

Mapping a Multivalued Attribute

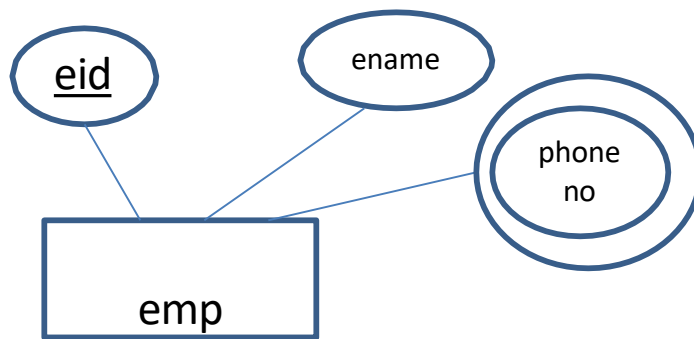
(a) **EMPLOYEE** entity type with multivalued attribute (**Skill**)



(b) Two resulting tables



- Convert the following ER diagram into relational schema

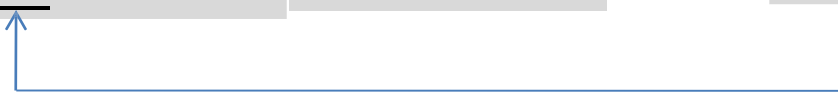


Relational schema

employee

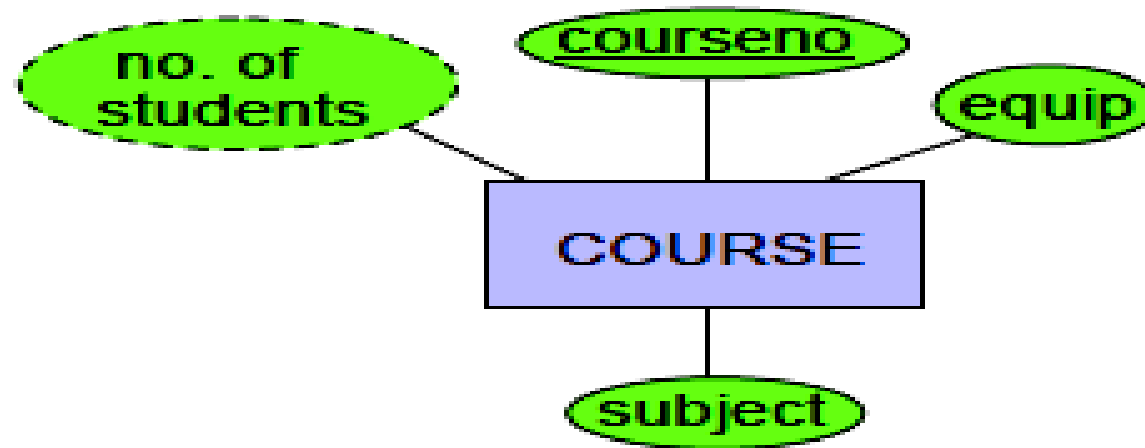


employee_phone



1-Mapping of Strong Entity Set

- Mapping a Derived attribute



COURSE

<u>courseno</u>	equip	subject
-----------------	-------	---------

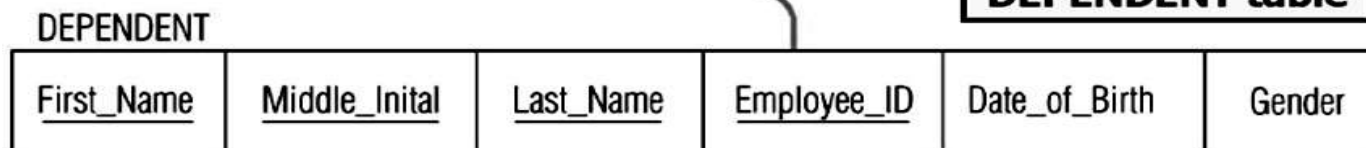
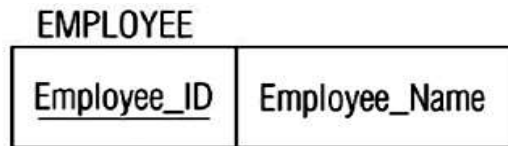
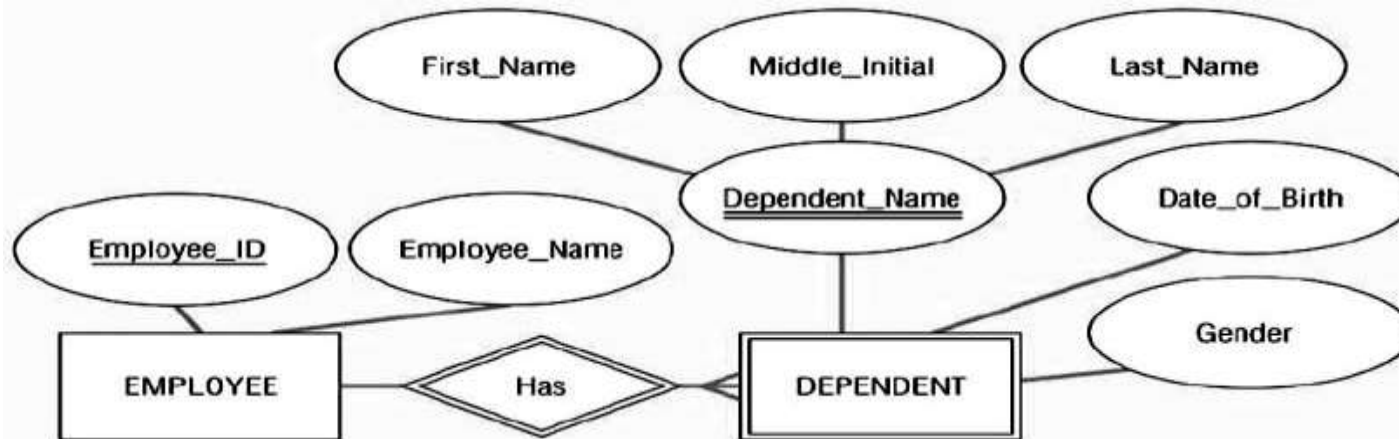
2-Mapping of Weak Entity Set

- For each weak entity set W with owner entity type E, create a **separate relation or table R** with all the attributes of W along with primary key of E as foreign key taken from the owner entity E.
- Primary key of R is composed of the discriminator and all the remaining attributes of the weak entity plus the primary key of the owner entity as foreign key of R on which it is existent dependent.

2-Mapping of Weak Entity Set

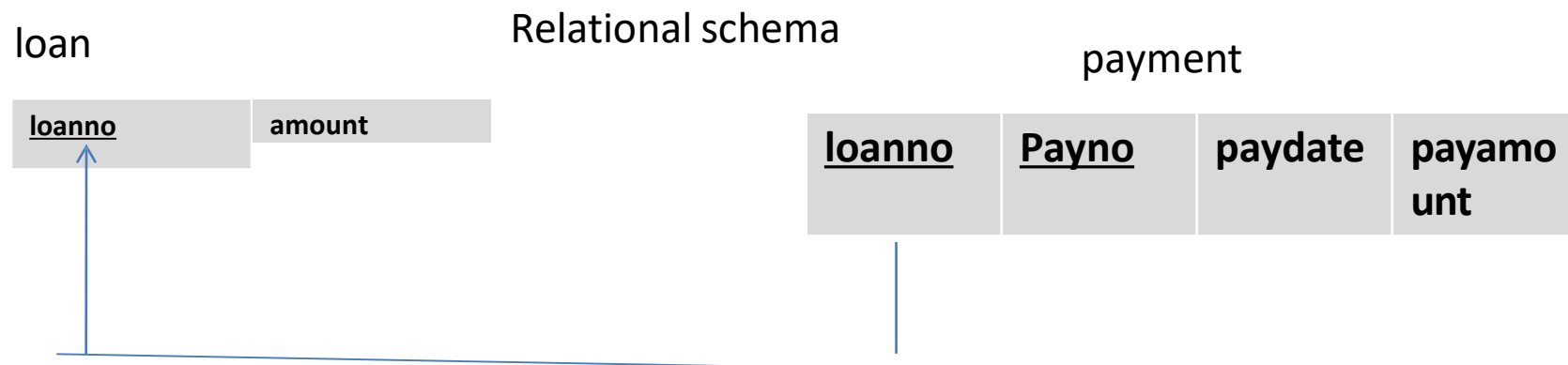
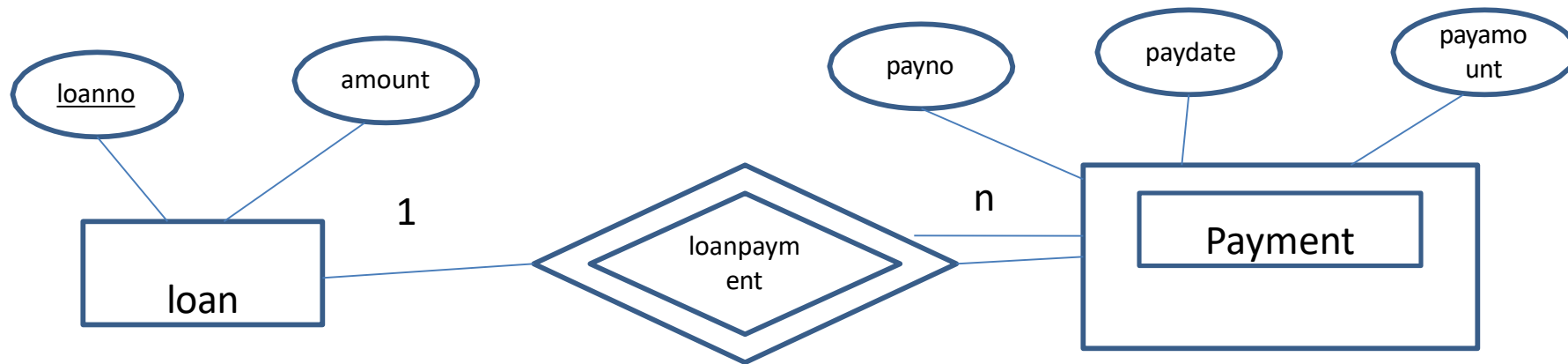
Example of mapping a weak entity

Weak entity **DEPENDENT**



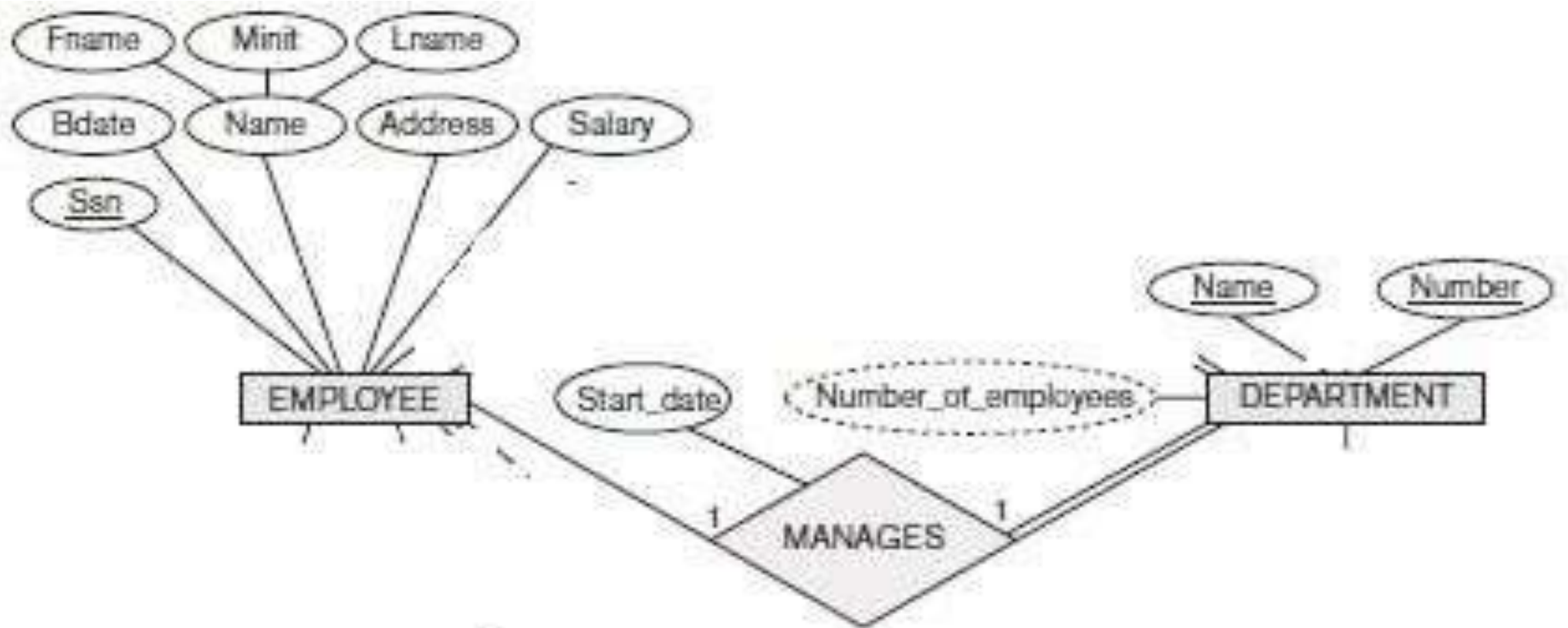
**Note the
Composite PK in
DEPENDENT table**

- Convert the following ER diagram into relational schema



3) Mapping a binary 1 : 1 relationship

- For each binary **1:1** relationship type **R** in the ER schema, identify participating entity type **S** and **T** of **R**.
- Add Primary key of one participating relation as the foreign key of the other participating relation
 - If only one participating type is total participation then primary key of the parital participation side becomes the foreign key of the total participation side.



3) Mapping a binary 1 : 1 relationship

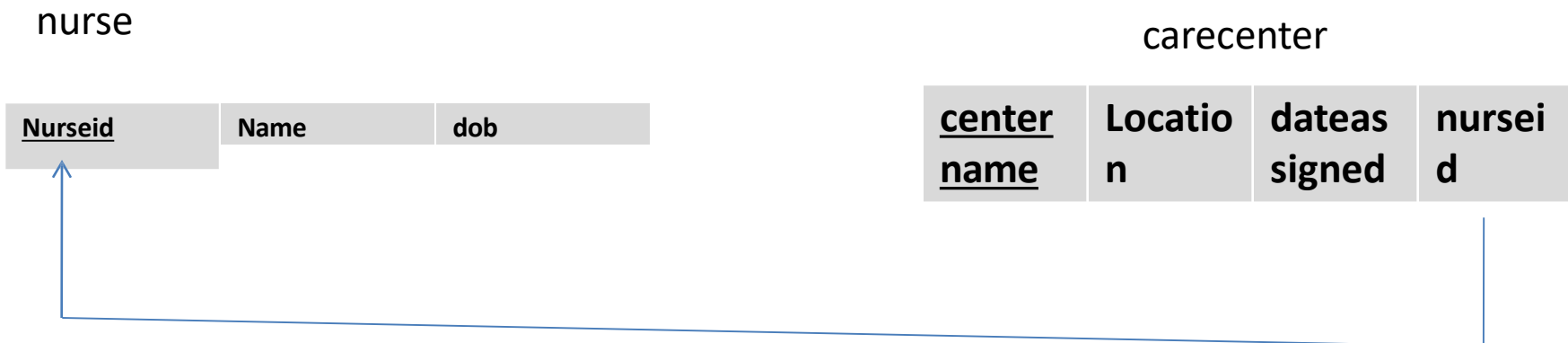
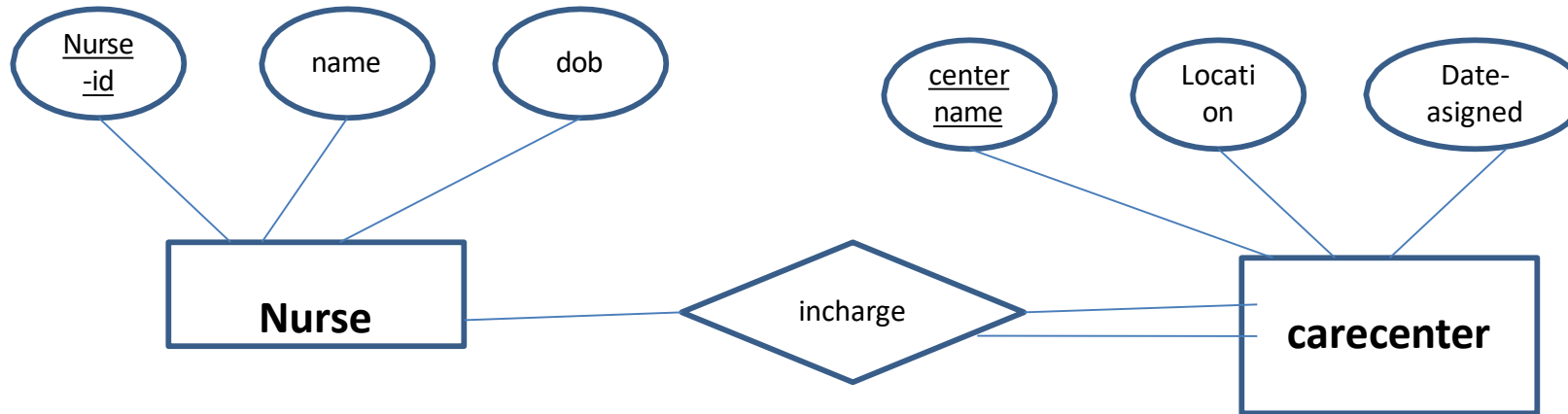
EMPLOYEE

<u>SSN</u>	Bdate	Fname	Minit	Lname	address	salary
------------	-------	-------	-------	-------	---------	--------

<u>Dnumber</u>	Name	Mgrssn	Startdate
----------------	------	--------	-----------

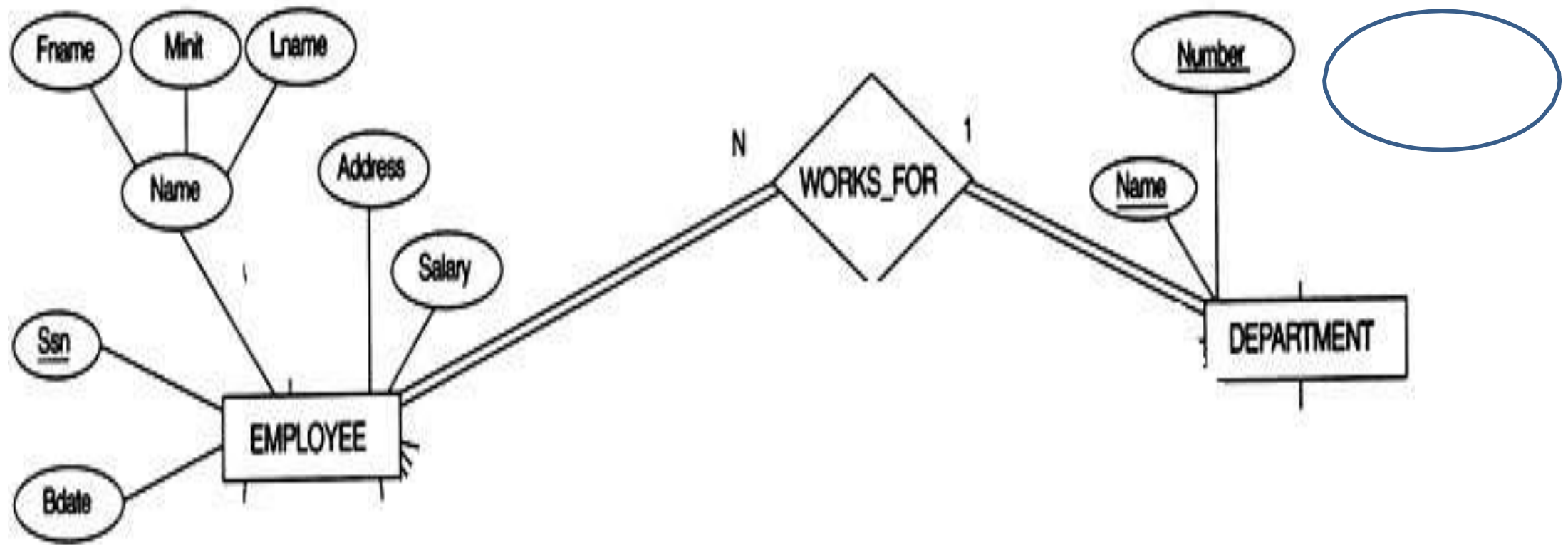
DEPARTMENT

3) Mapping a binary 1 : 1 relationship



4) Mapping a binary 1 : N relationship

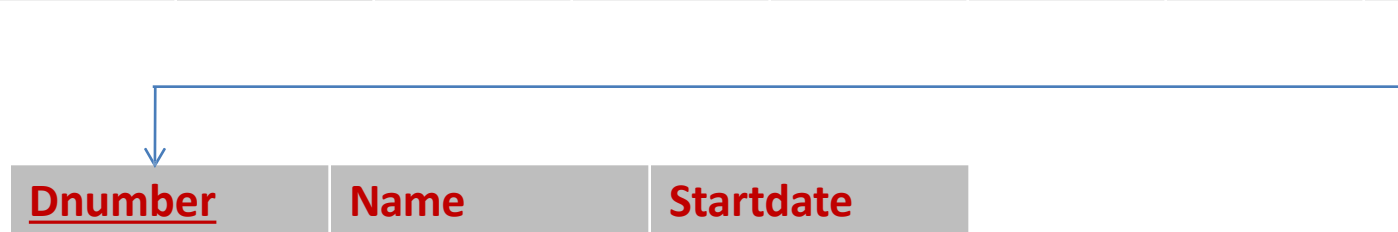
- For each binary **1:N** relationship type **R** in the ER schema, identify participating entity type **S** at **N side**.
- Include the primary key of T (entity type at **1 side**) as the foreign key of S(many side).
- Include the descriptive attribute of R in the table **S**.



4) Mapping a binary 1 : N relationship

EMPLOYEE

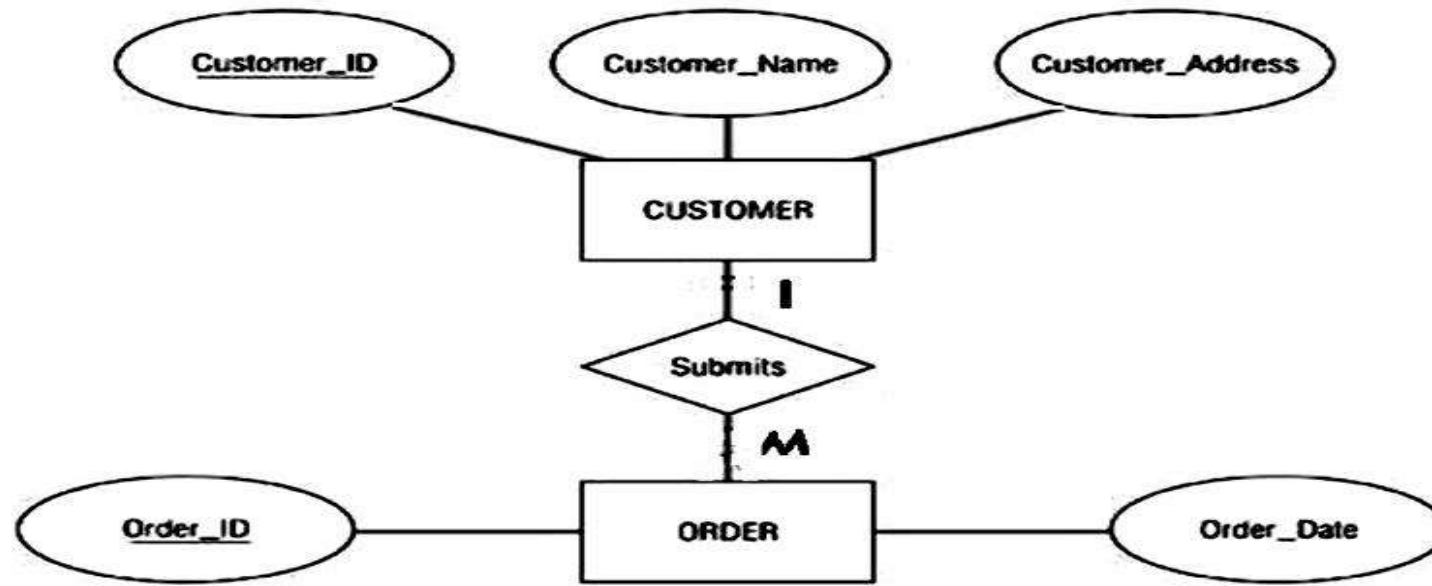
<u>SSN</u>	Bdate	Fname	Minit	Lname	address	salary	DNO
------------	-------	-------	-------	-------	---------	--------	-----



DEPARTMENT

<u>Dnumber</u>	Name	Startdate
----------------	------	-----------

4) Mapping a binary 1 : N relationship



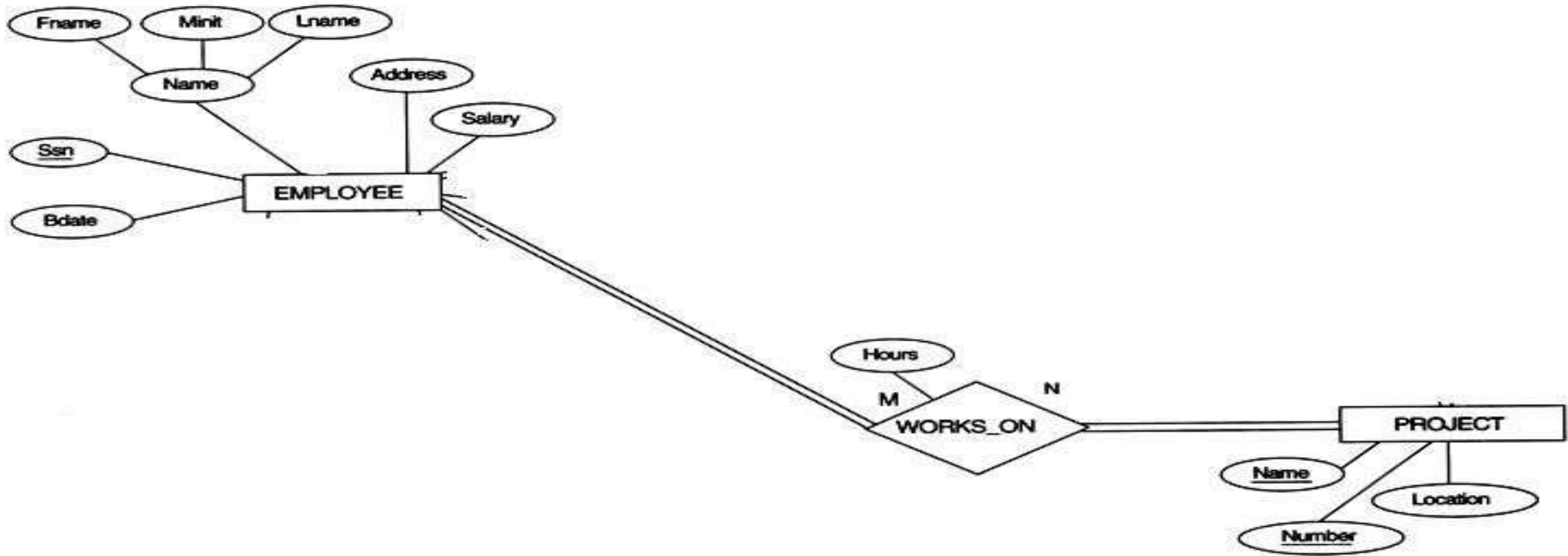
customer

order

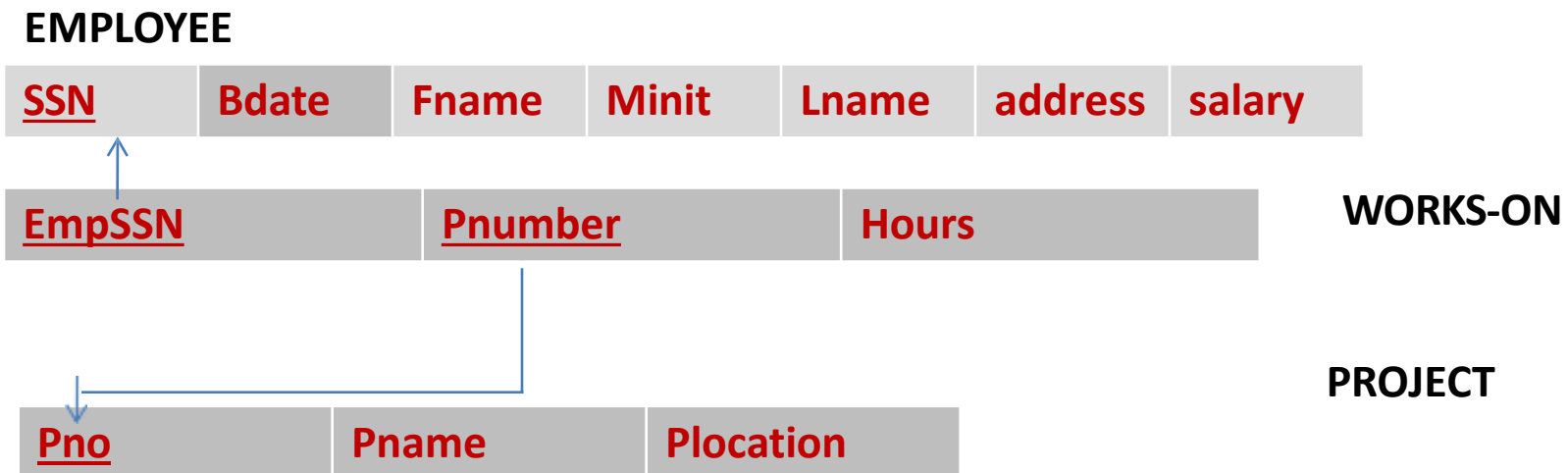


5) Mapping a binary $M : N$ relationship

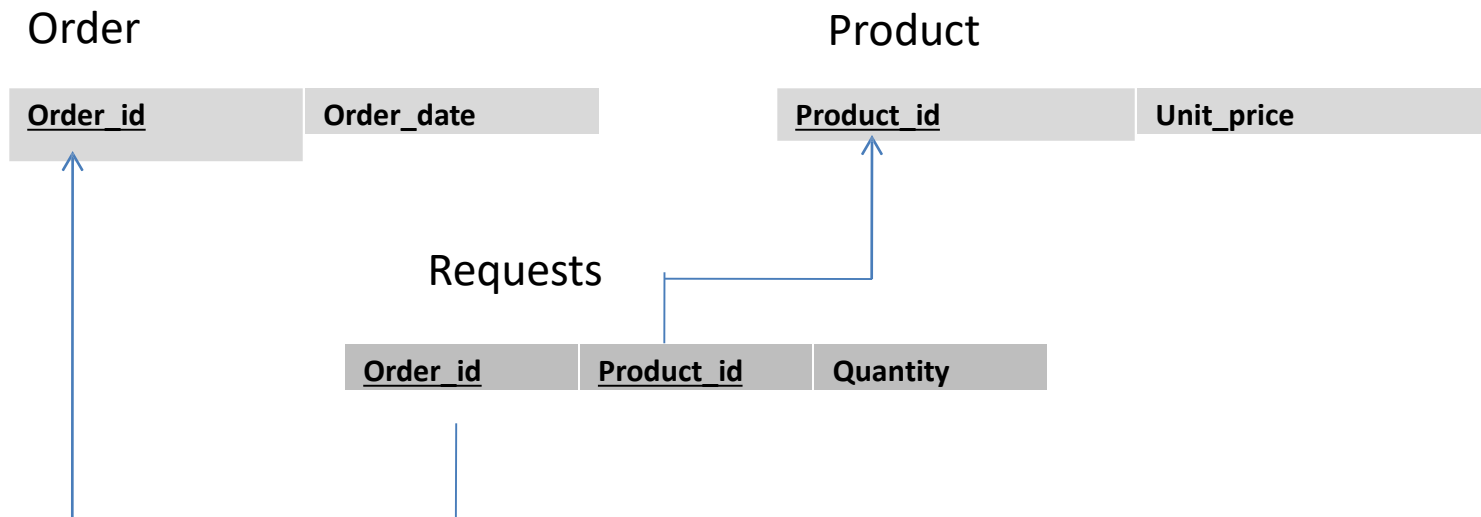
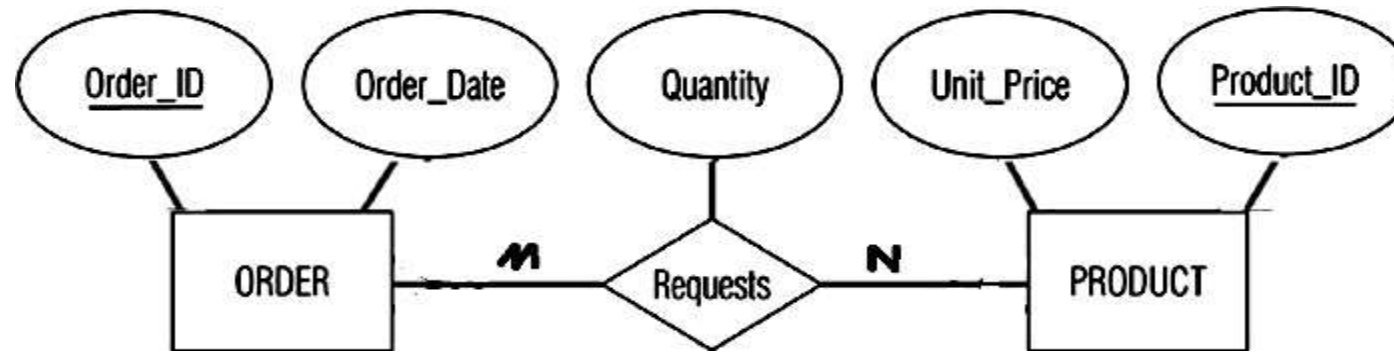
- For each binary **M:N** relationship type **R** with participating entity type **S** and **T**, creates a new relation or table **R1** to represent **R**.
- Include primary key of S and T as the foreign key of R1.
- **Primary key** of **R1** is the combination of primary key of R and S.
- Include the descriptive attribute of **R** in the table R1.



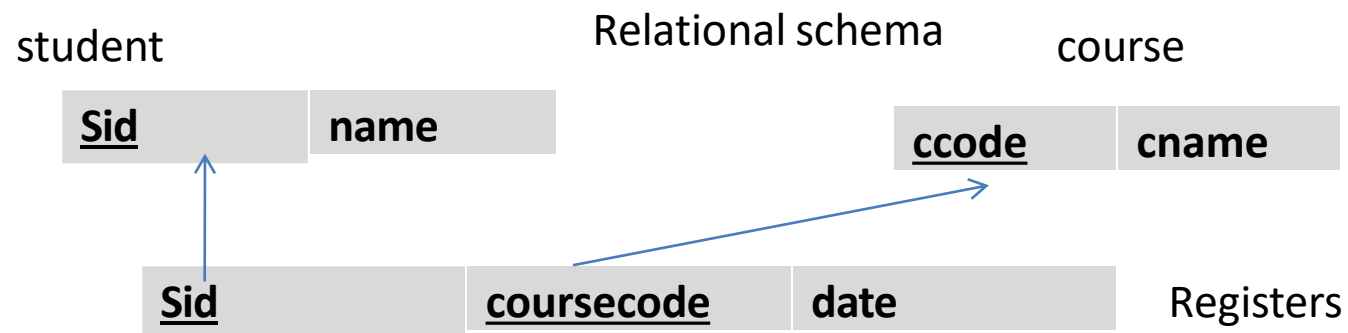
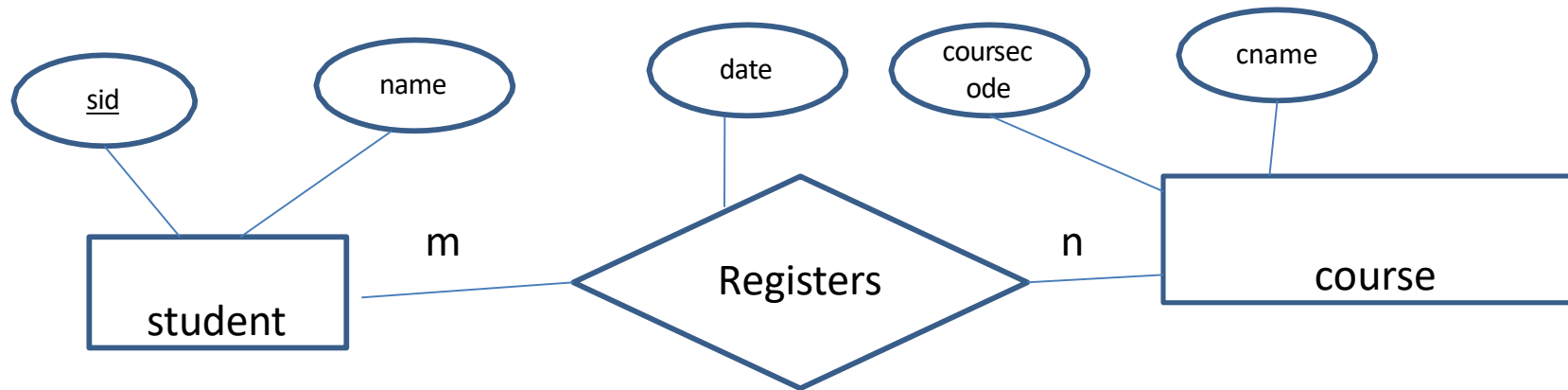
5) Mapping a binary M : N relationship



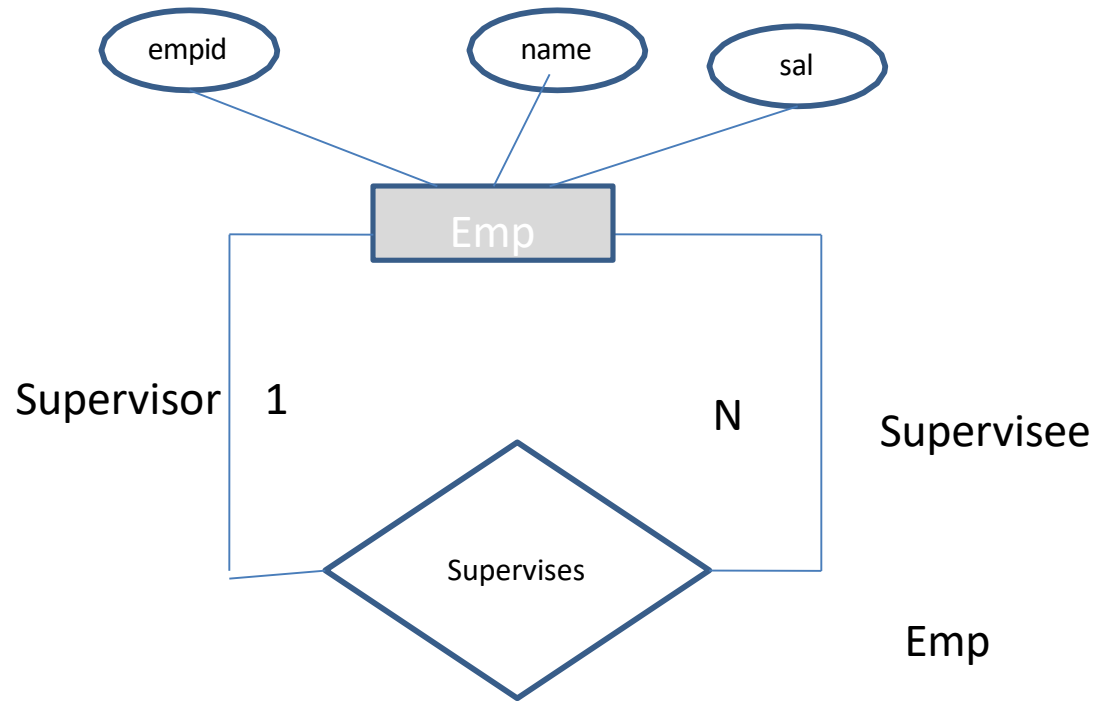
5) Mapping a binary M : N relationship



- Convert the following ER diagram into relational schema



5) Mapping a recursive relationship



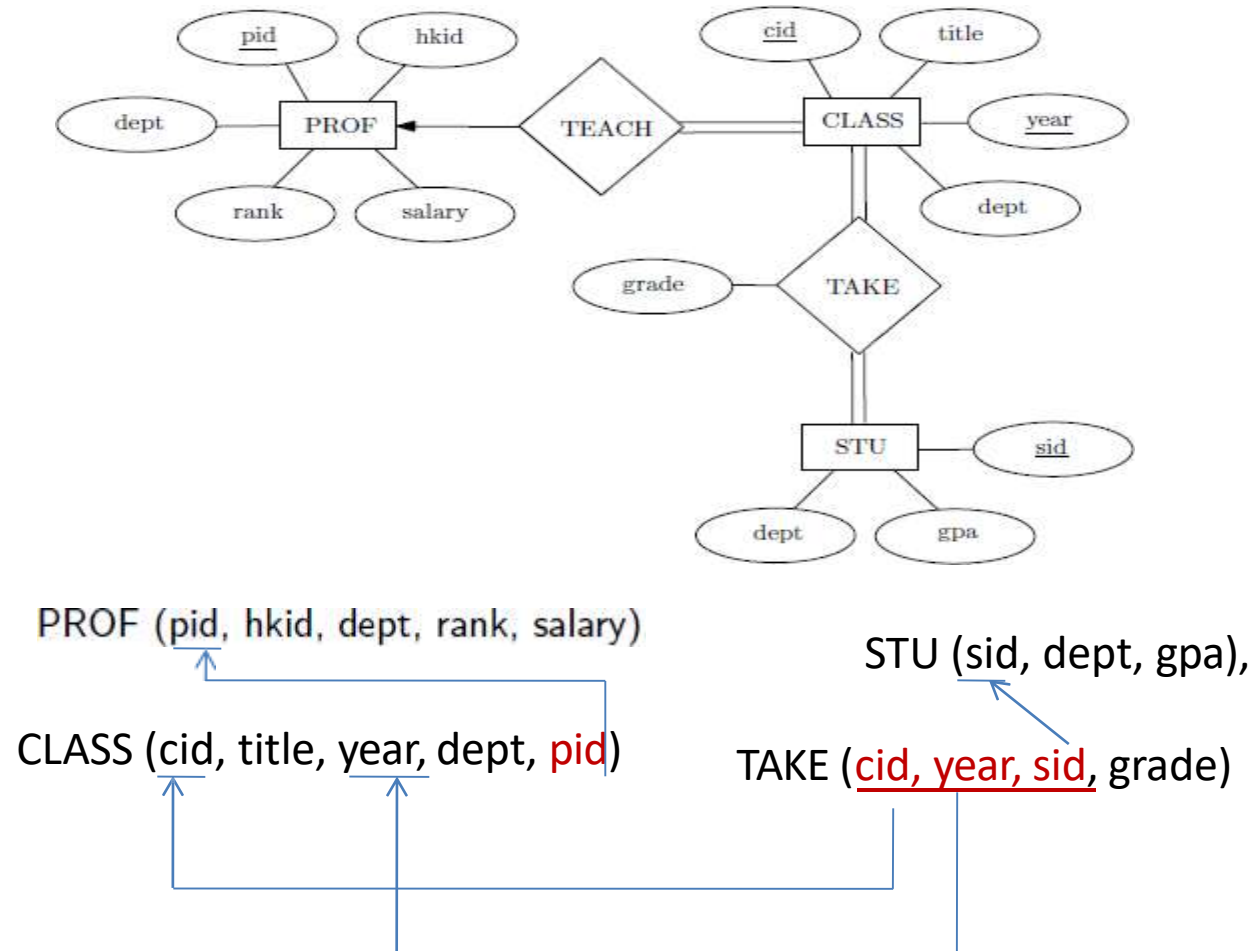
Relational schema

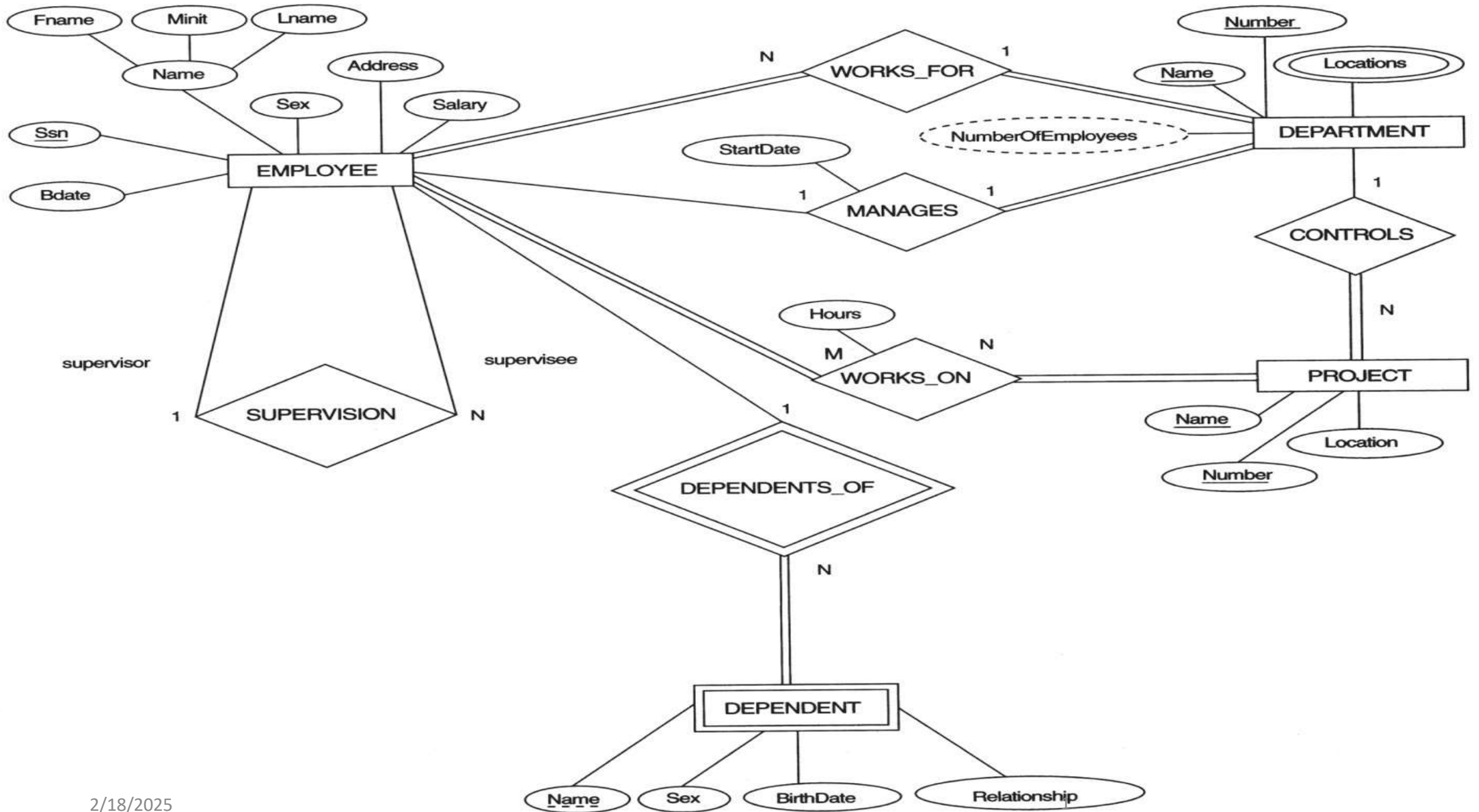
Emp

empid	Name	sal	supvid
-------	------	-----	--------

A diagram showing a self-referencing arrow on the **supvid** attribute of the **Emp** table. The arrow starts from the **supvid** attribute and points back to the **empid** attribute, indicating a recursive relationship.

Example





Example

EMPLOYEE

Fname	Minit	Lname	<u>Ssn</u>	Bdate	Address	Sex	Salary	Super_ssn	Dno
-------	-------	-------	------------	-------	---------	-----	--------	-----------	-----

DEPARTMENT

Dname	<u>Dnumber</u>	Mgr_ssn	Mgr_start_date
-------	----------------	---------	----------------

DEPT_LOCATIONS

<u>Dnumber</u>	<u>Dlocation</u>
----------------	------------------

PROJECT

Pname	<u>Pnumber</u>	Plocation	Dnum
-------	----------------	-----------	------

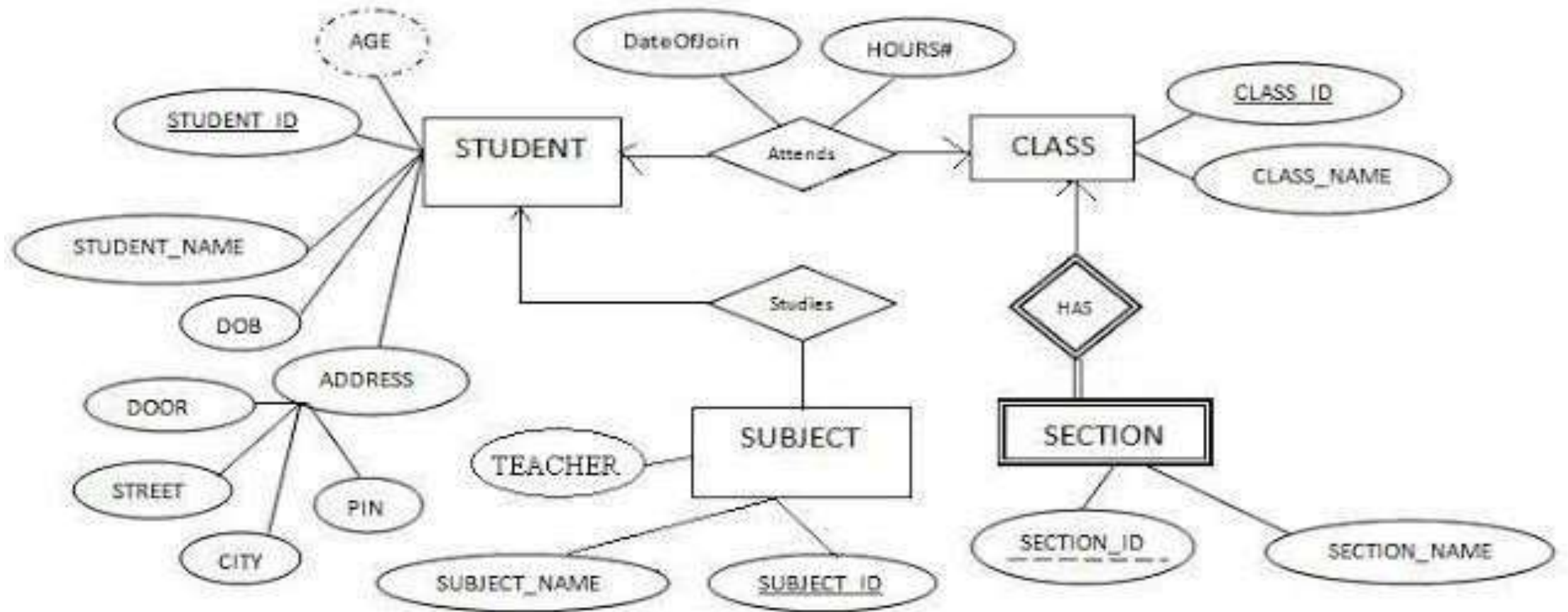
WORKS_ON

<u>Essn</u>	<u>Pno</u>	Hours
-------------	------------	-------

DEPENDENT

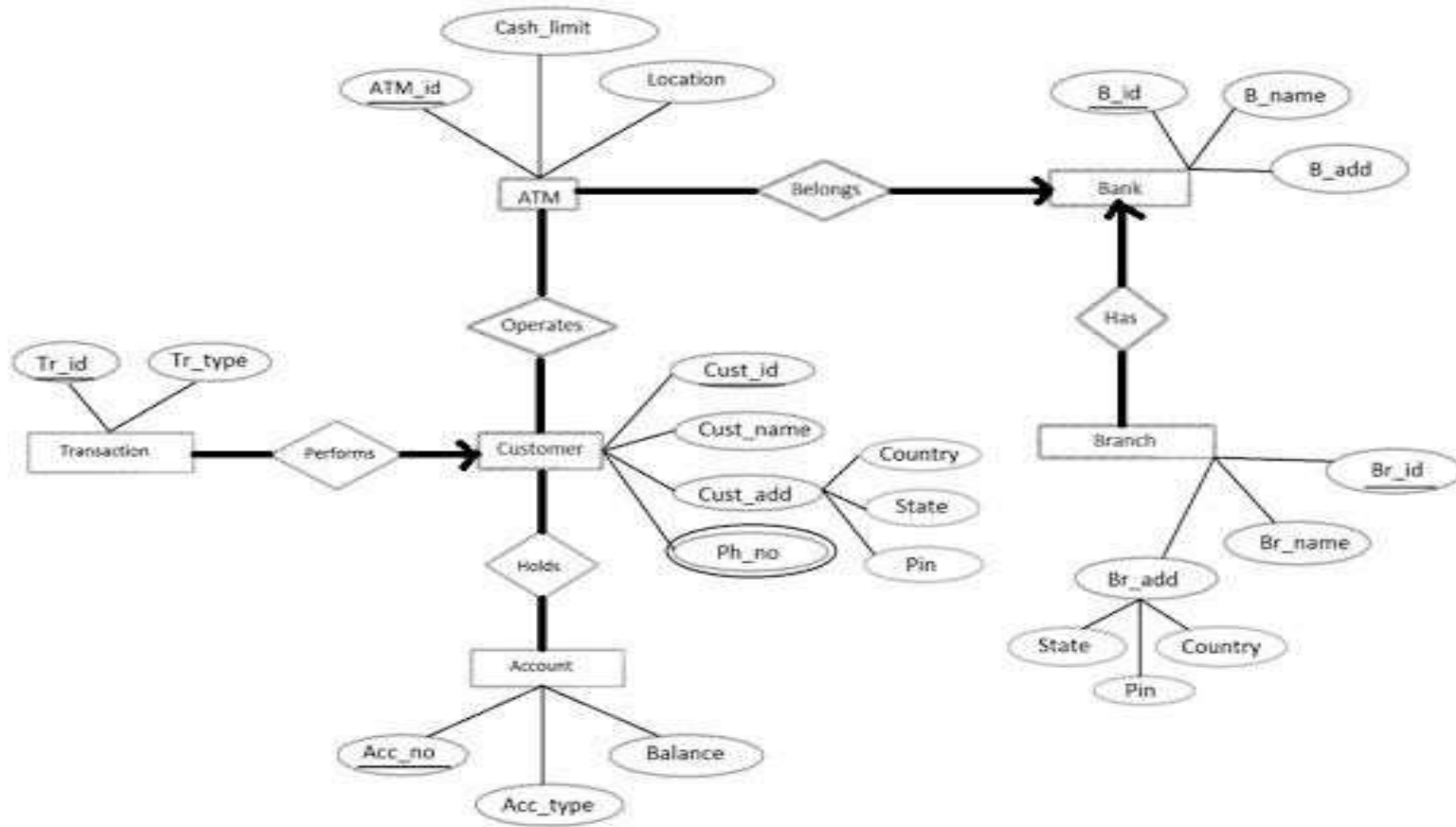
<u>Essn</u>	<u>Dependent_name</u>	Sex	Bdate	Relationship
-------------	-----------------------	-----	-------	--------------

Example



- STUDENT (Student_ID, Student_Name, DOB, Door, Street, City, Pin, Class-id, DateOfJoin, Hours#)
- CLASS (Class_ID, Class_Name)
- SUBJECT (Subject_ID, Subject_Name, Teacher, **Student_ID**)
- SECTION (Section_ID, **Class_ID**, Section_Name)

Example



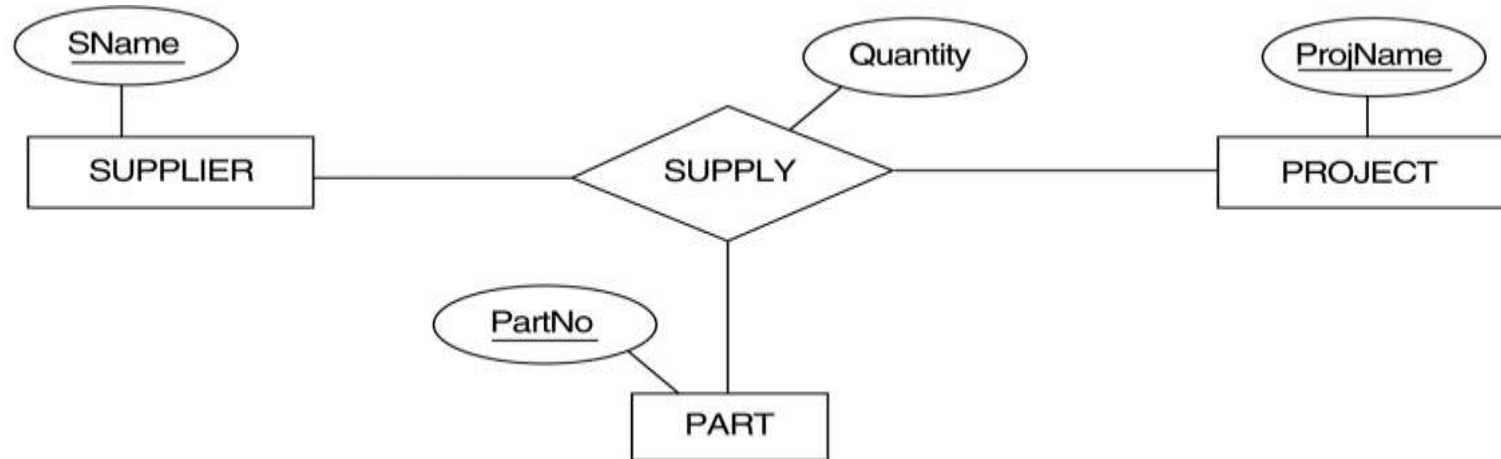
Example

- Bank (B_id, B_name, B_add)
- Customer (Cust_id, Cust_name, State, Country, Pin)
- Customer_Phone (Cust_id, Ph_no)
- Account (Acc_no, Acc_type, Balance)
- ATM (ATM_id, cash_limit, location, B_id)
- Branch (Br_id, Br_name, State, Country, Pin, B_id)
- Transaction (Tr_id, Tr_type, Cust_id)
- Operates (ATM_id, Cust_id)
- Holds (Cust_id, Acc_no)

Step 6: Mapping of N-ary Relationship Types.

- For each n-ary relationship type R, where $n > 2$, create a new relationship S to represent R.
- Include as foreign key attributes in S the primary keys of the relations that represent the participating entity types.
- Also include any simple attributes of the n-ary relationship type (or simple components of composite attributes) as attributes of S.

(a)



SUPPLIER

<u>SNAME</u>	...
--------------	-----

PROJECT

<u>PROJNAME</u>	...
-----------------	-----

PART

<u>PARTNO</u>	...
---------------	-----

SUPPLY

<u>SNAME</u>	<u>PROJNAME</u>	<u>PARTNO</u>	QUANTITY
--------------	-----------------	---------------	----------

ASSIGNMENTS

Convert the ER model to Relational model

